

NLP

Transformer

Основао на [NLP Course | For You](#)

План

- seq2seq
- attention
- transformer
- BERT
- attention performance
- finetuning

Sequence-to-sequence



Задача

Human Translation

$$y^* = \arg \max_y p(y|x)$$

The “probability” is intuitive and is given by a human translator’s expertise

Machine Translation

model parameters

$$y' = \arg \max_y p(y|x, \theta)$$

Questions we need to answer

- **modeling**

How does the model for $p(y|x, \theta)$ look like?

- **learning**

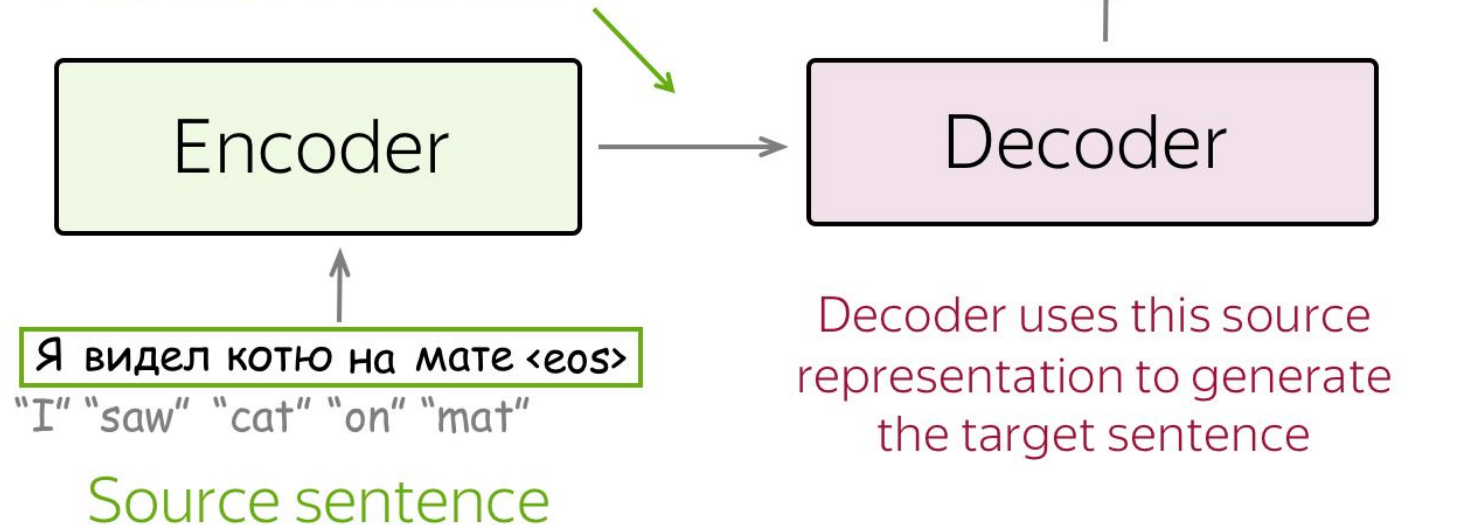
How to find θ ?

- **search**

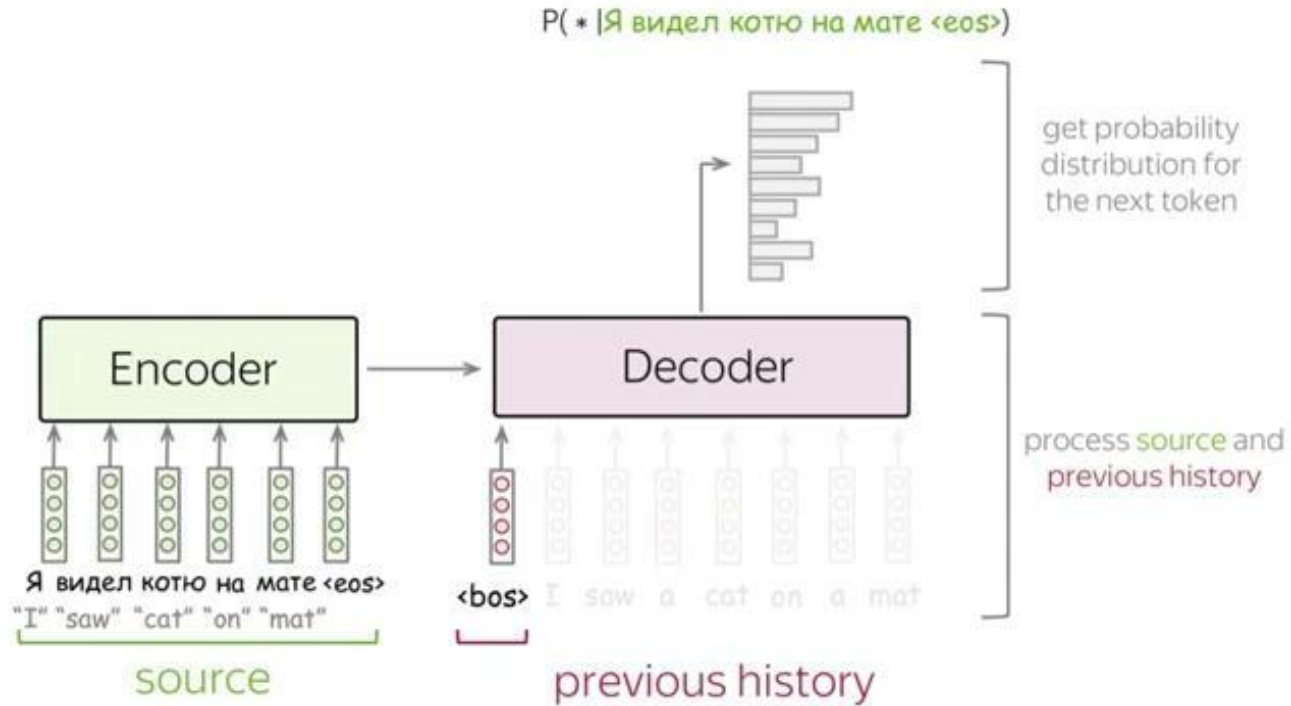
How to find the argmax?

Encoder-decoder

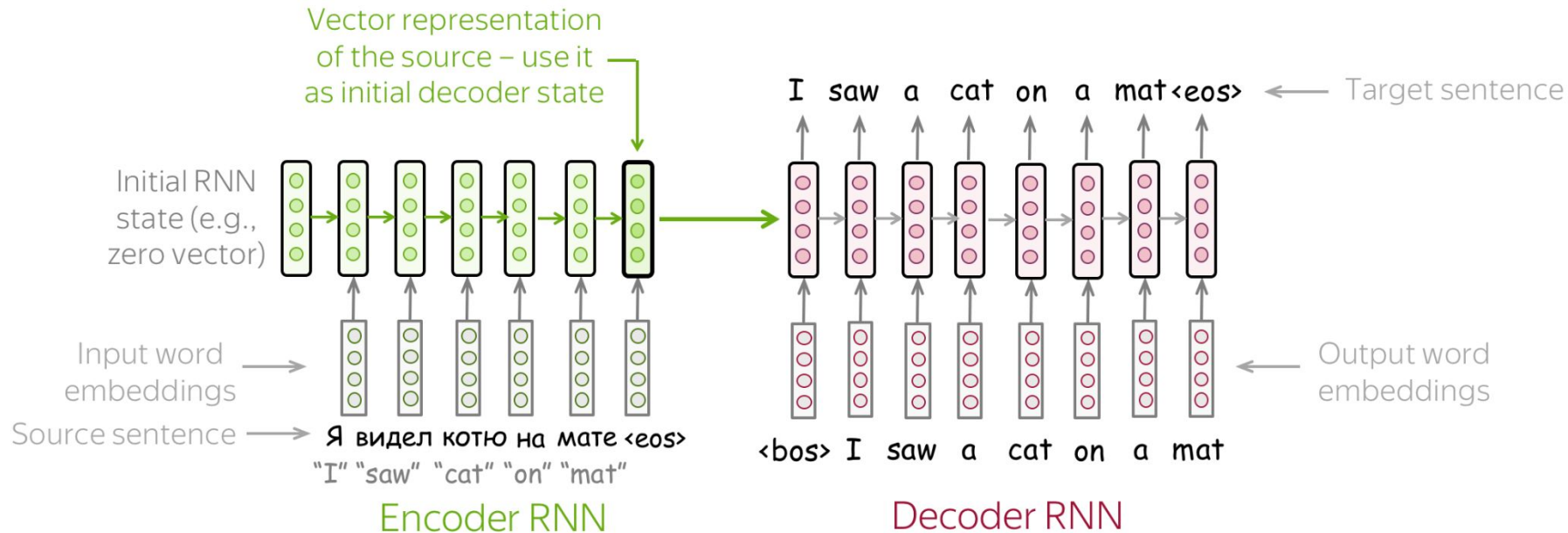
Encoder builds a representation of the source and gives it to the decoder



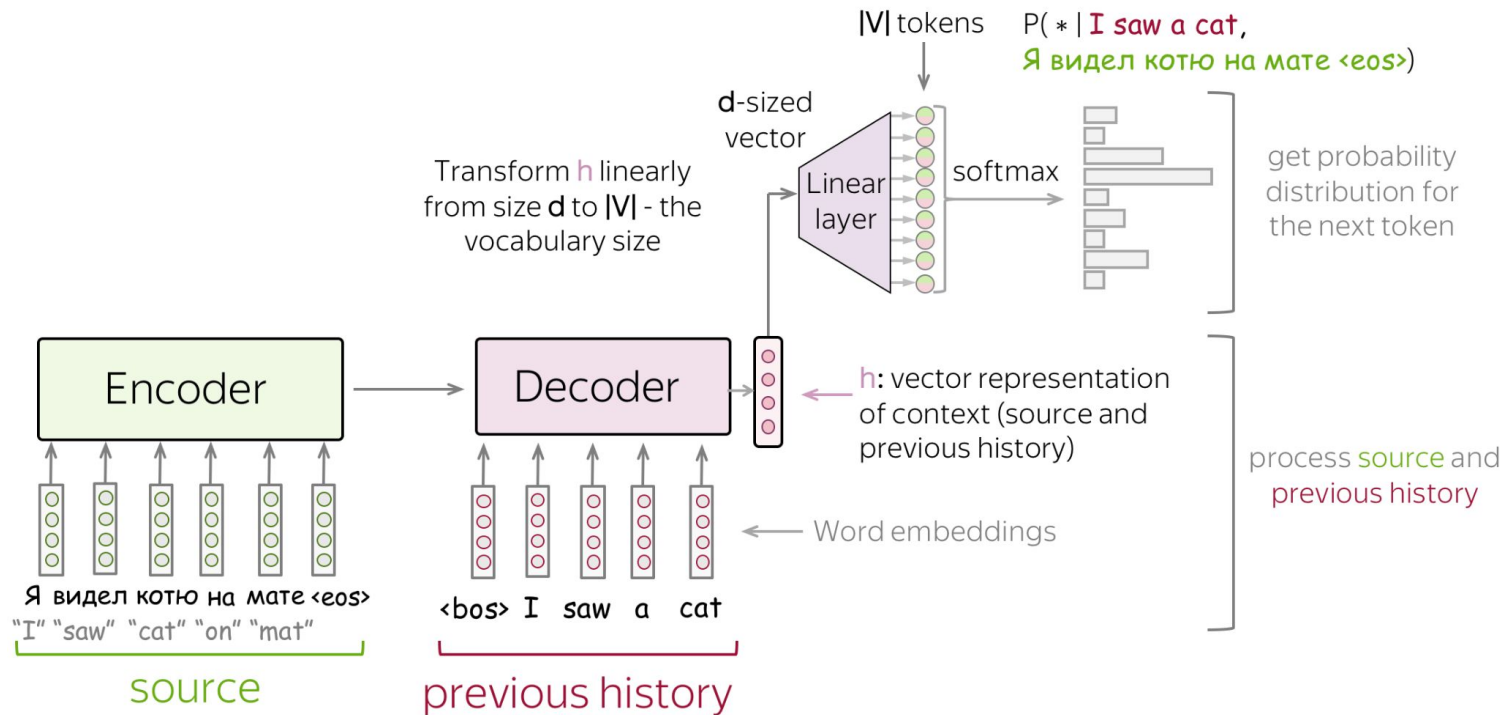
Encoder-decoder



RNN



Как генерируем?



Как генерируем?

Source sequence:

Я видел котю на мате <eos>

"I" "saw" "cat" "on" "mat"

Target sequence:

I saw a cat on a mat <eos>

previous tokens

we want the model
to predict this

← one training example

← one step for this example

Model prediction: $p(* | \text{I saw a, Я ... <eos>})$



← cat →

Target



Loss = $-\log(p(\text{cat})) \rightarrow \min$



decrease

increase

decrease

Как найти argmax?

$$y' = \arg \max_y p(y|x) = \arg \max_y \prod_{t=1}^n p(y_t | y_{<t}, x)$$

How to find the argmax?

Жадный подход

$$\arg \max_y \prod_{t=1}^n p(y_t | y_{<t}, x) \neq \prod_{t=1}^n \arg \max_{y_t} p(y_t | y_{<t}, x)$$

Beam search

<bos>

Start with the begin of sentence token or with an empty sequence

Once again генерация

Encoder: read source



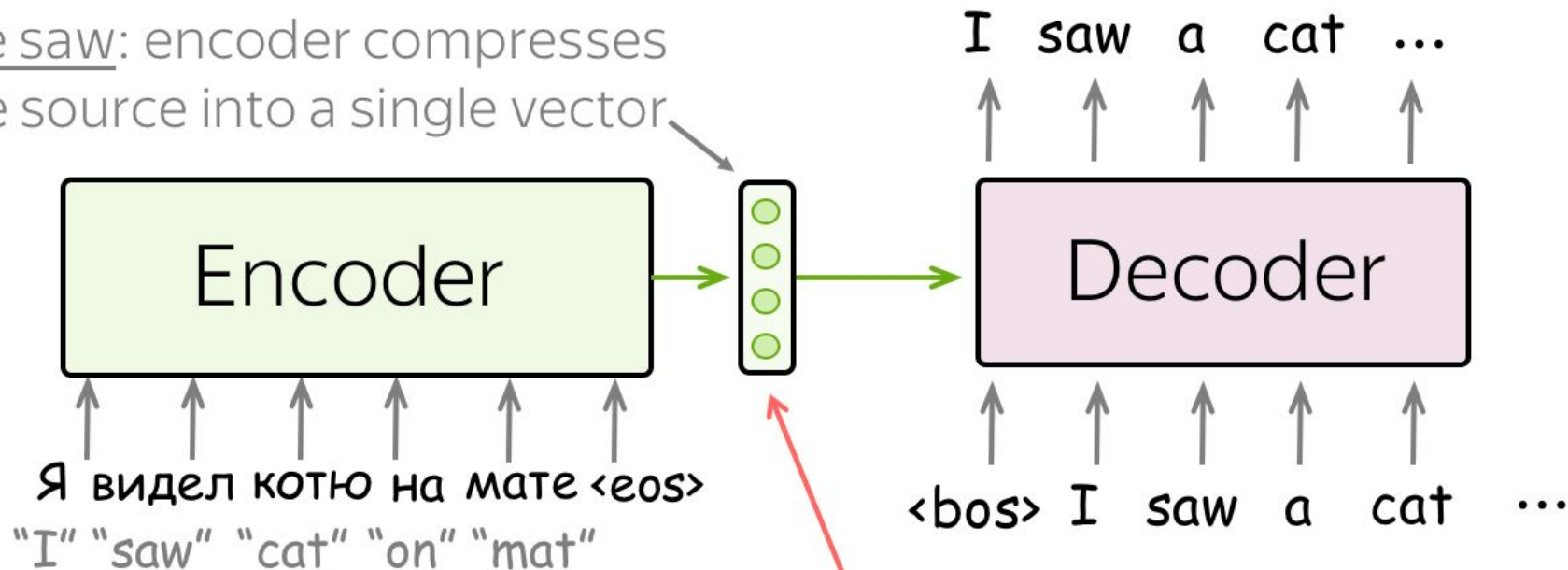
we are here

Source: Я видел котю на мате <eos>
"I" "saw" "cat" "on" "mat"

Target: I saw a cat on a mat <eos>

Но есть нюанс

We saw: encoder compresses the source into a single vector



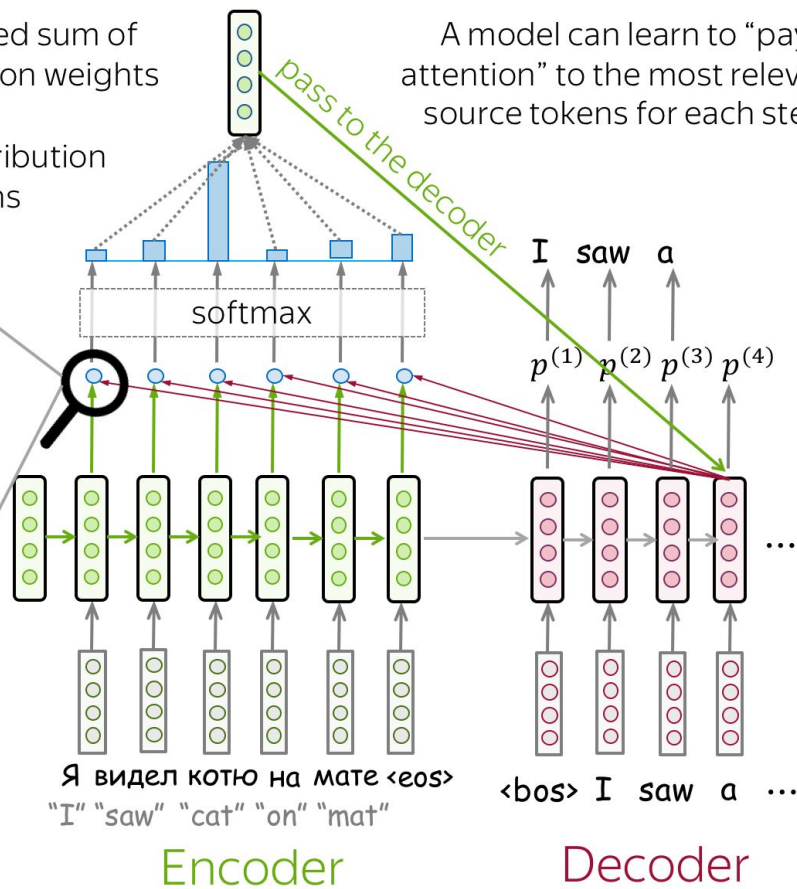
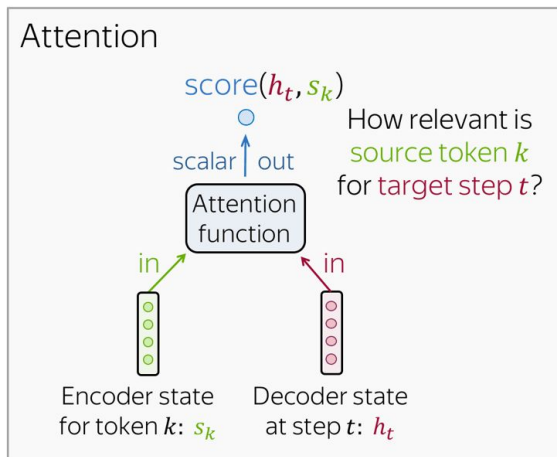
Problem: this is a bottleneck!

Attention

Attention output: weighted sum of encoder states with attention weights

Attention weights: distribution over source tokens

A model can learn to “pay attention” to the most relevant source tokens for each step



Подсчёт

Attention output

↑
(weighted
sum)

Attention weights

↑
(softmax)

Attention scores

↑

Attention input

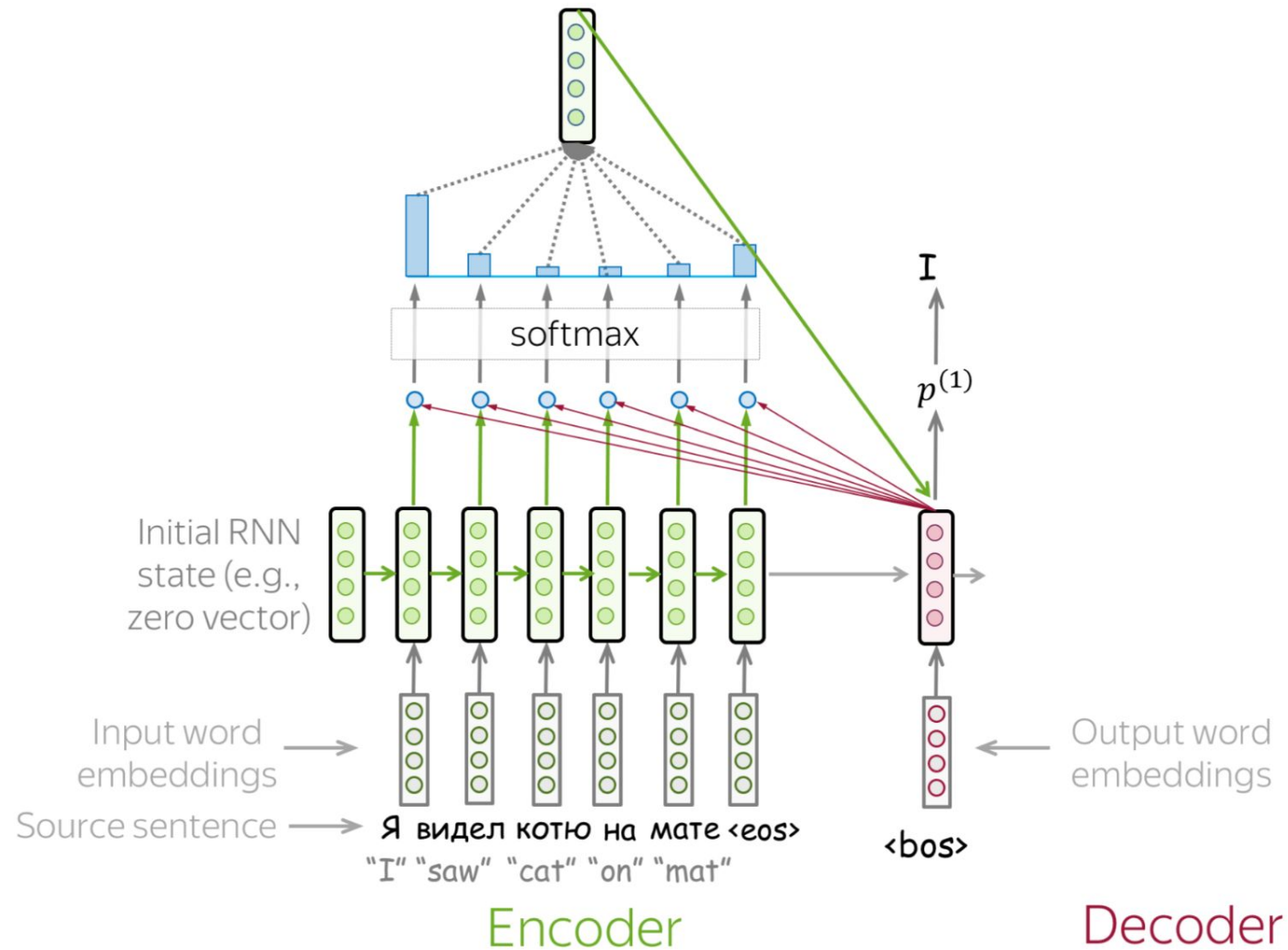
$$\underset{\substack{\uparrow \\ \text{"source context for decoder step } t"}}}{c^{(t)}} = a_1^{(t)} s_1 + a_2^{(t)} s_2 + \dots + a_m^{(t)} s_m = \sum_{k=1}^m a_k^{(t)} s_k$$

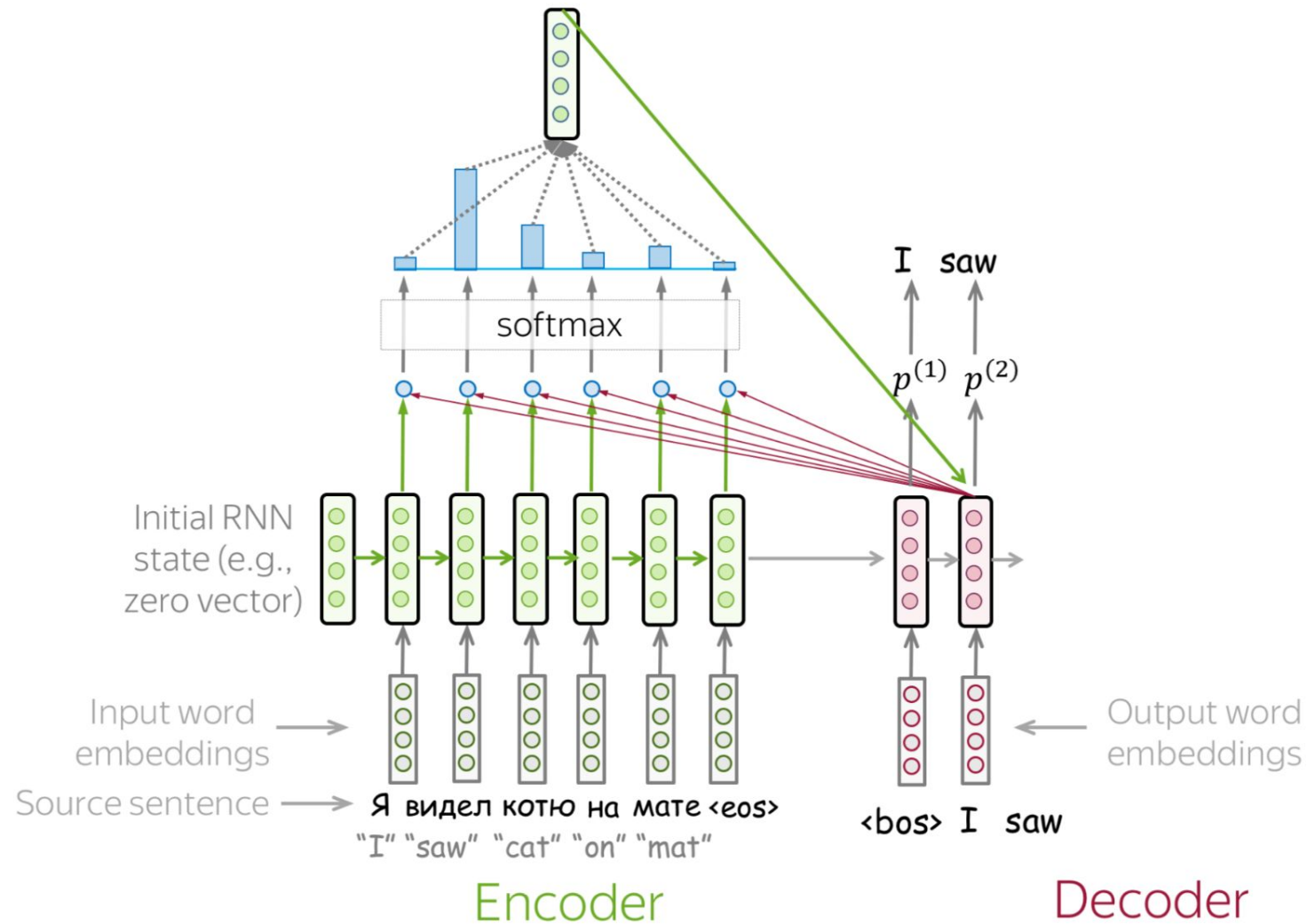
$$\underset{\substack{\uparrow \\ \text{"attention weight for source token } k \text{ at decoder step } t"}}}{a_k^{(t)}} = \frac{\exp(\text{score}(h_t, s_k))}{\sum_{i=1}^m \exp(\text{score}(h_t, s_i))}, k = 1..m$$

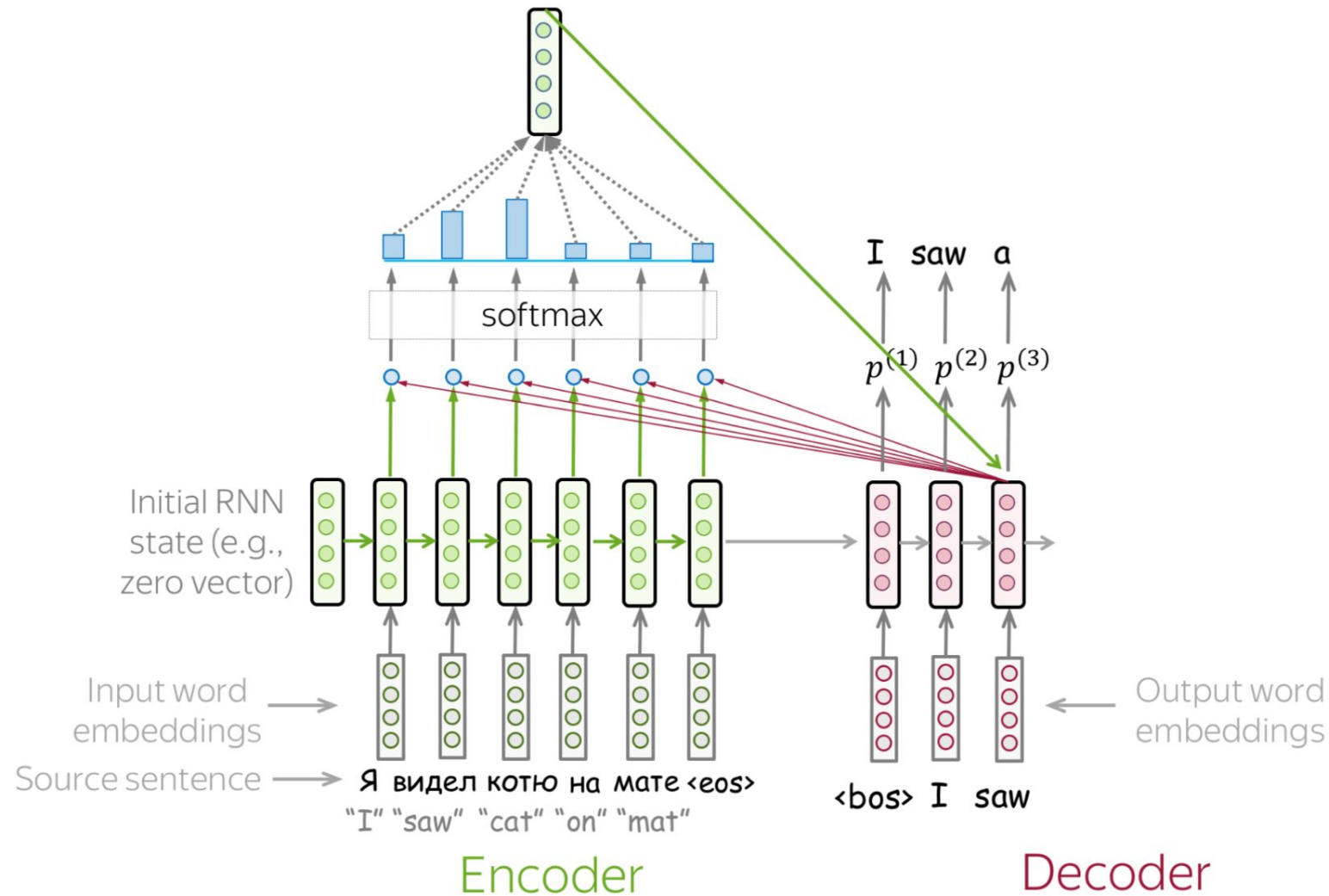
$$\underset{\substack{\uparrow \\ \text{"How relevant is source token } k \text{ for target step } t?"}}}{\text{score}(h_t, s_k)}, k = 1..m$$

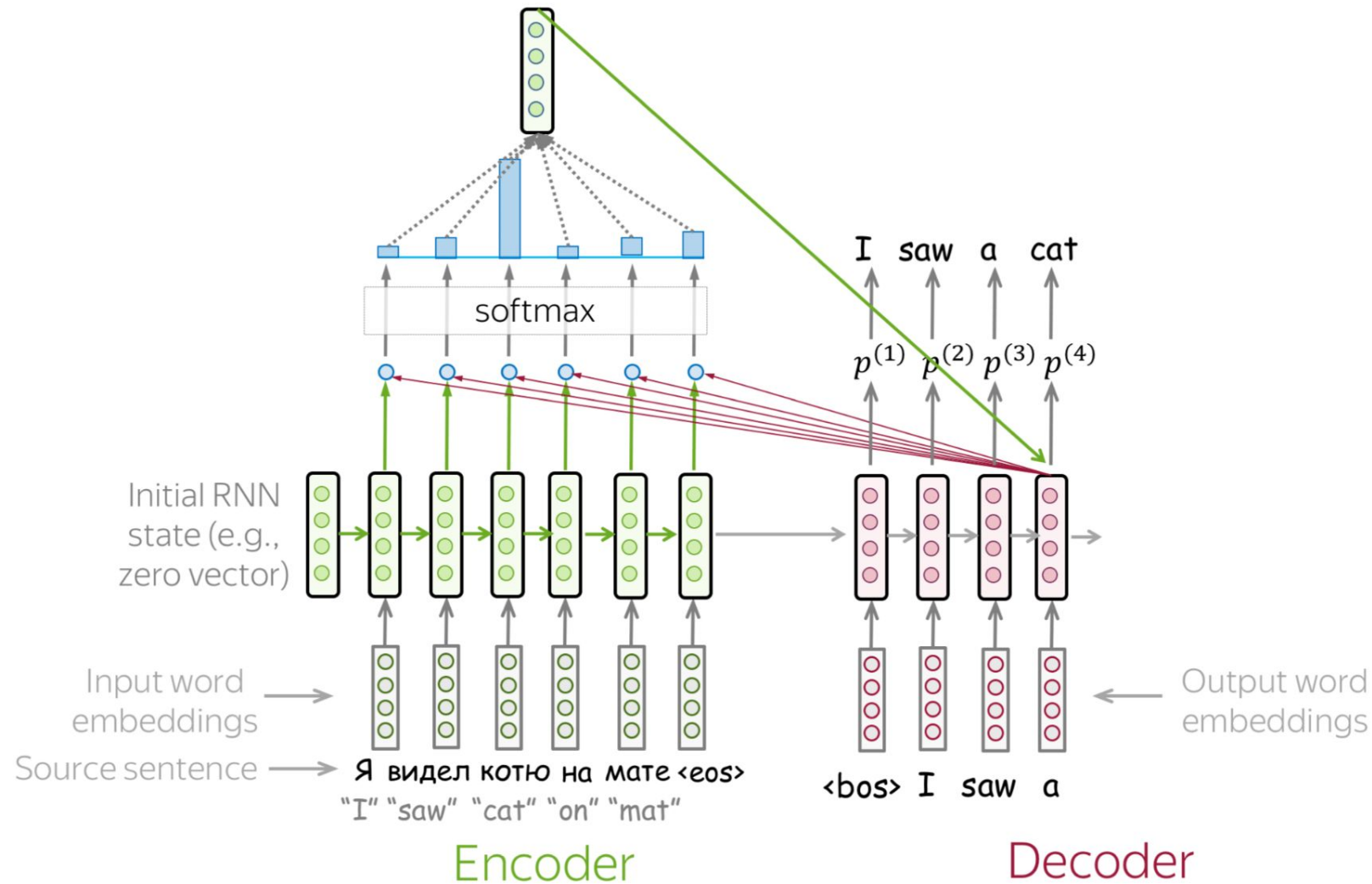
$s_1, s_2, \dots, s_m$
all encoder states

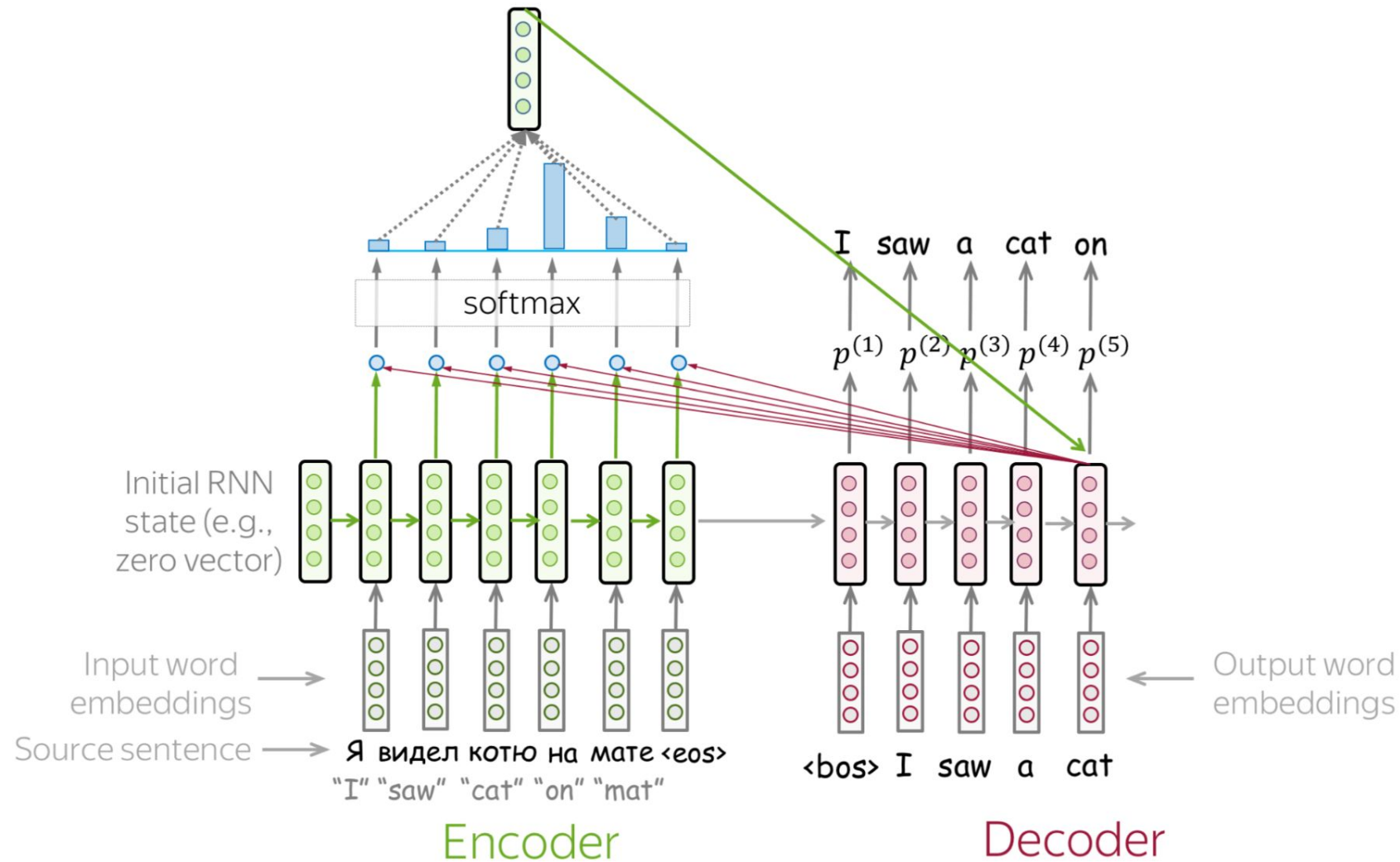
h_t
one decoder state

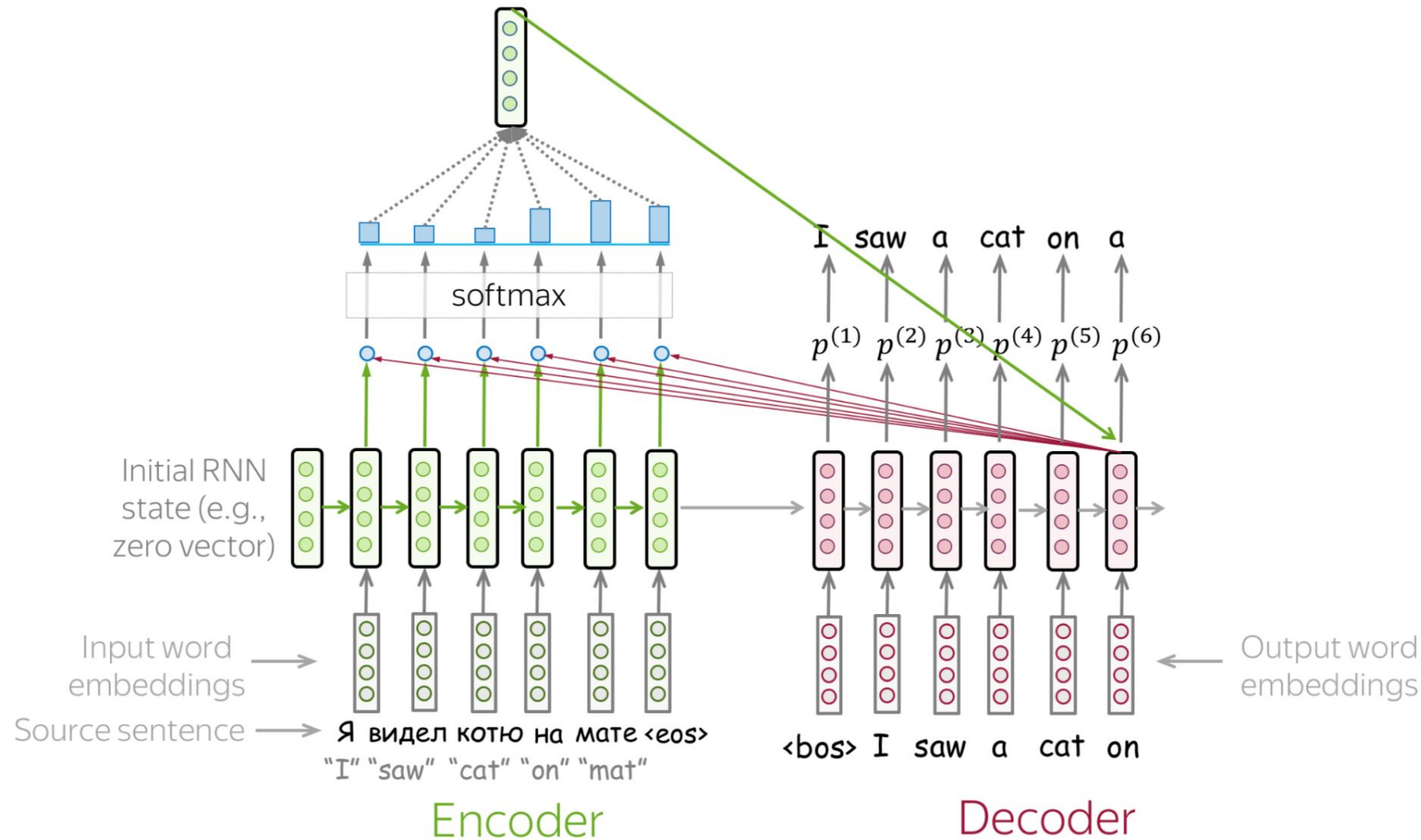


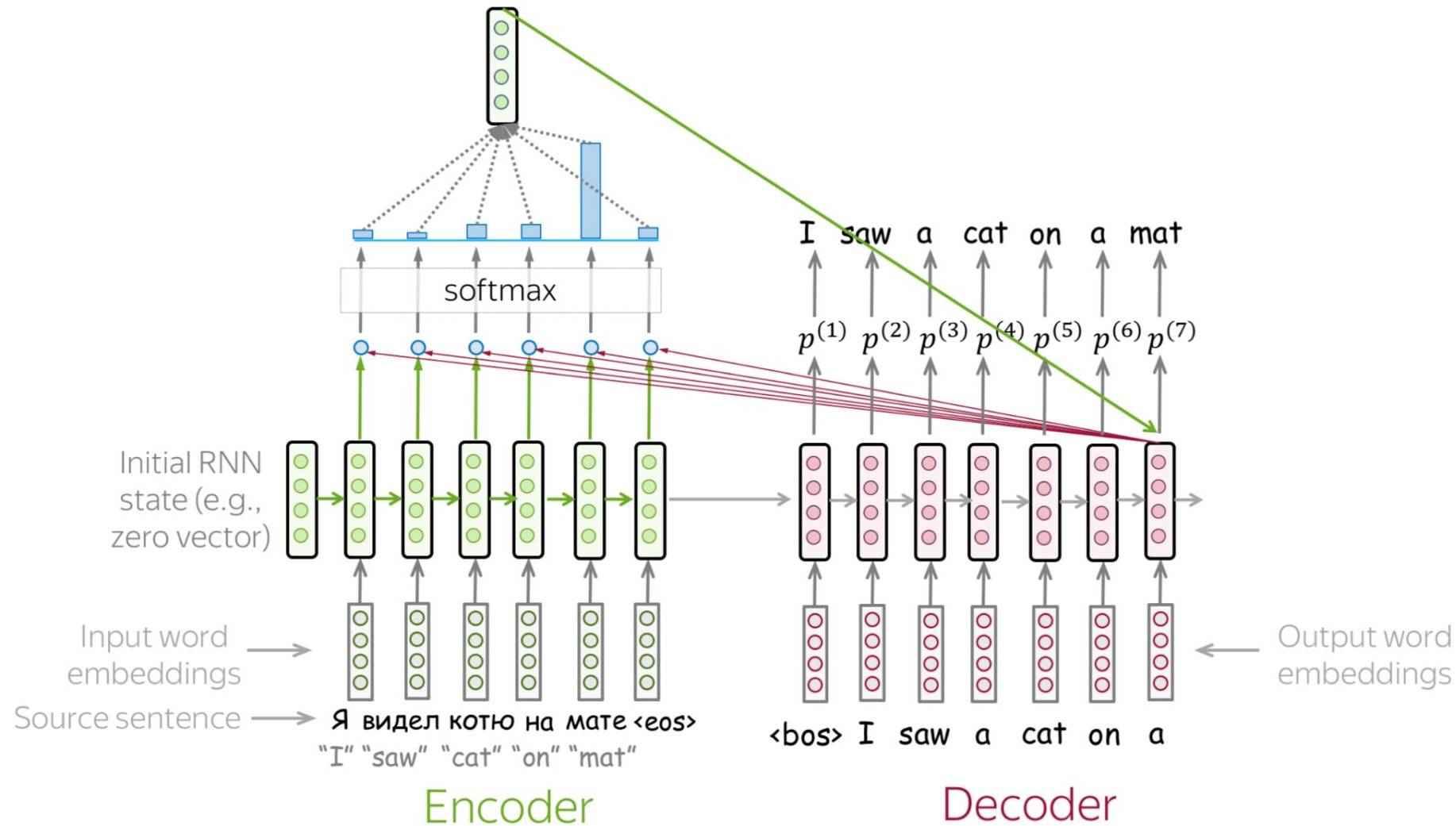


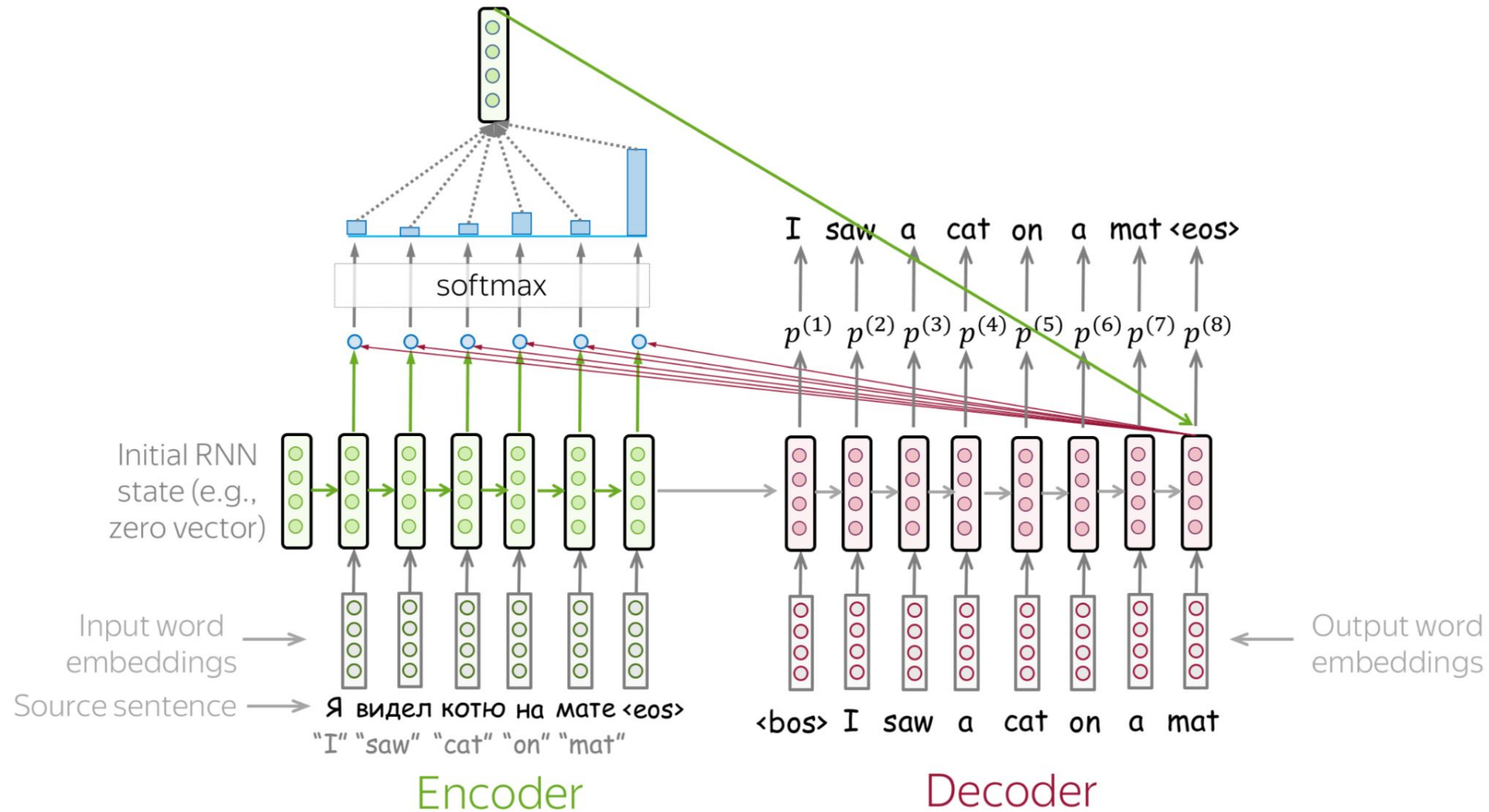




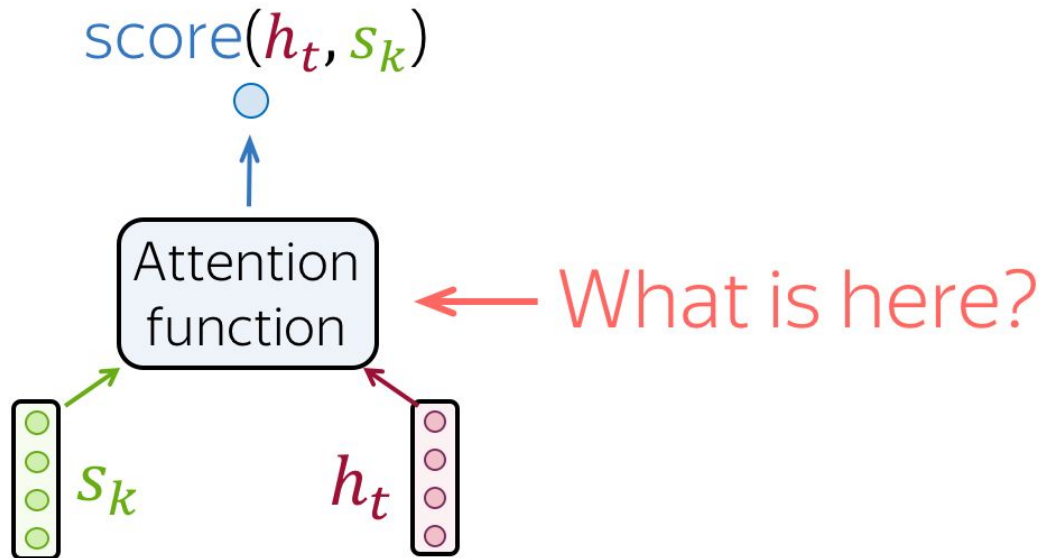






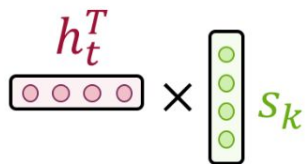


А что за функция?



В целом любая

Dot-product


$$h_t^T \times s_k$$

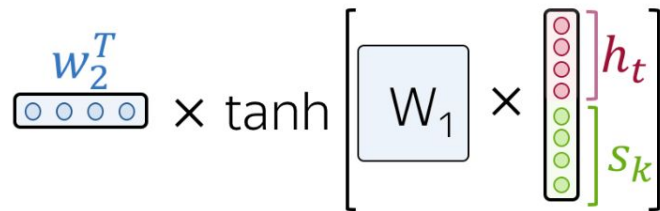
$$\text{score}(h_t, s_k) = h_t^T s_k$$

Bilinear


$$h_t^T \times [W] \times s_k$$

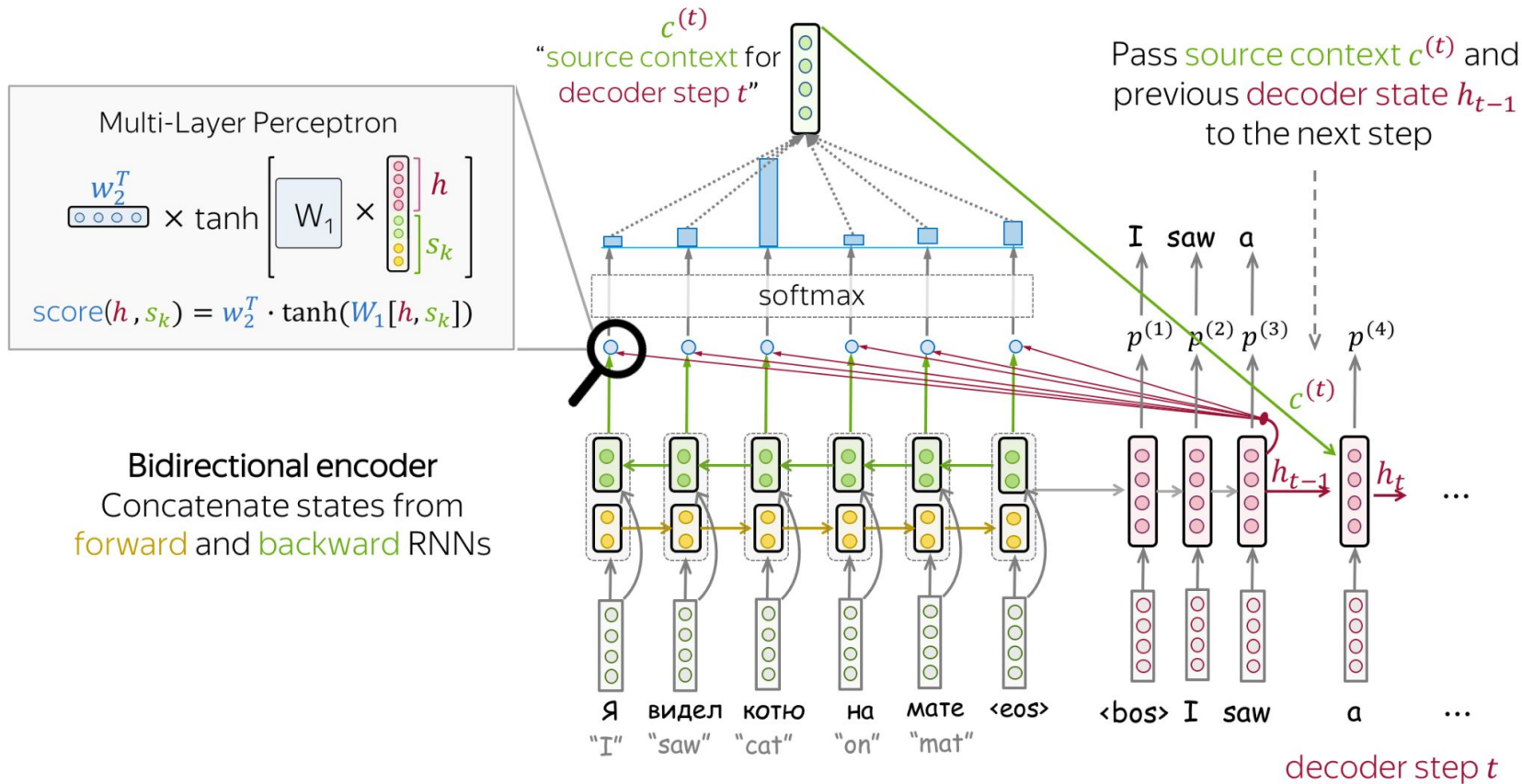
$$\text{score}(h_t, s_k) = h_t^T W s_k$$

Multi-Layer Perceptron

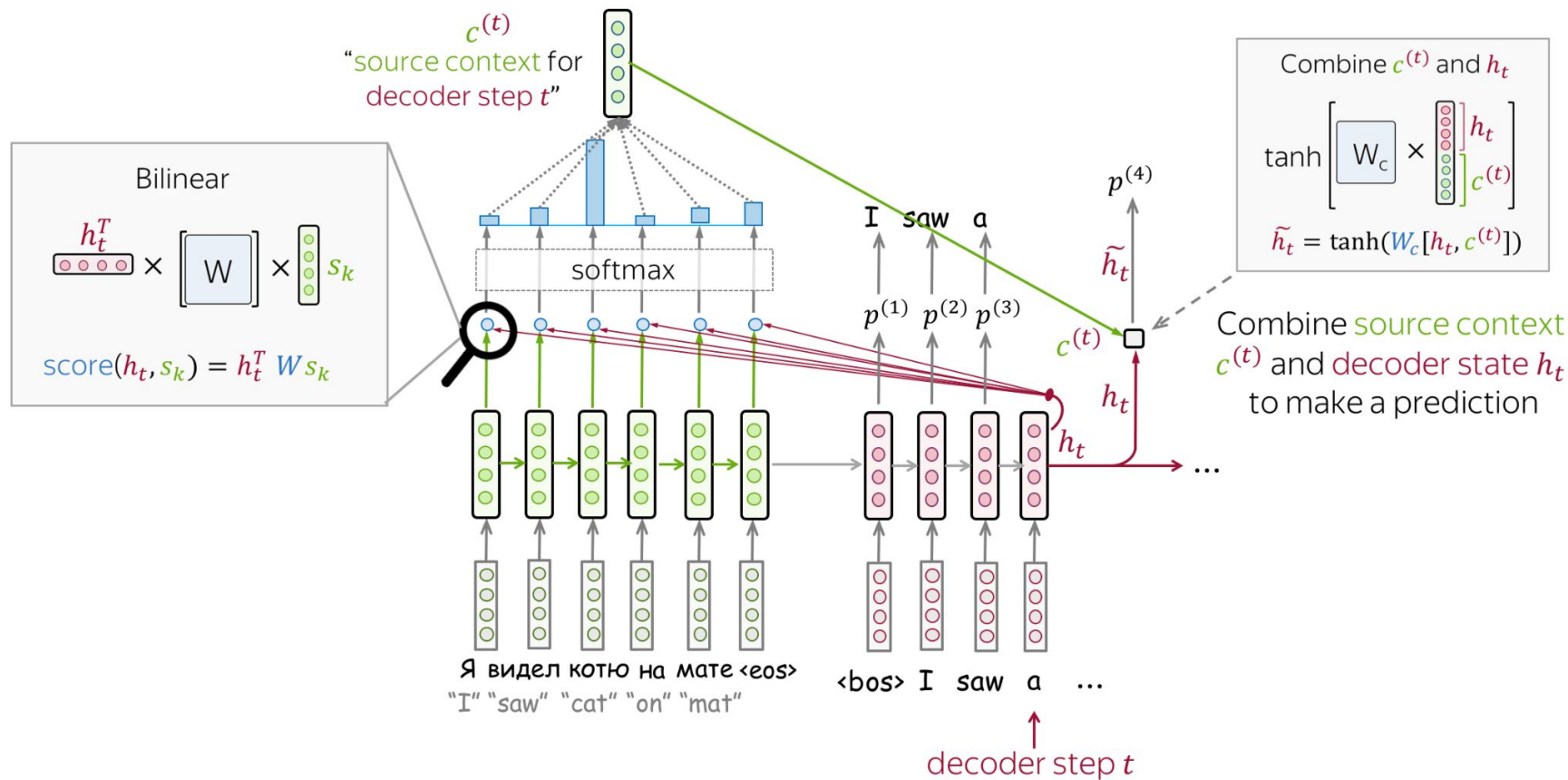

$$w_2^T \times \tanh \left[W_1 \times \begin{bmatrix} h_t \\ s_k \end{bmatrix} \right]$$

$$\text{score}(h_t, s_k) = w_2^T \cdot \tanh(W_1 [h_t, s_k])$$

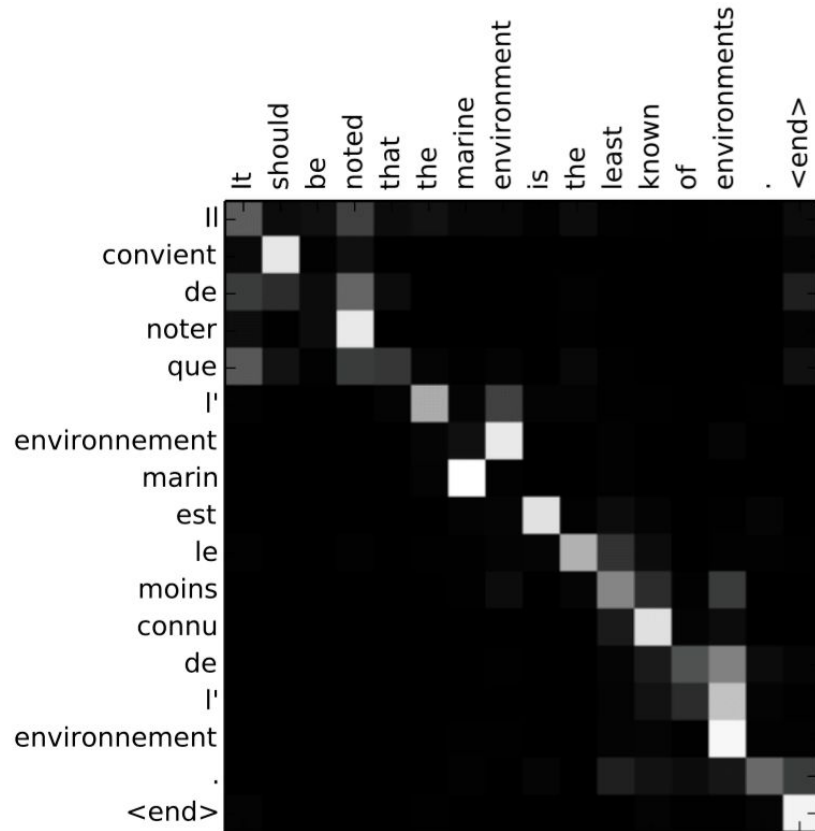
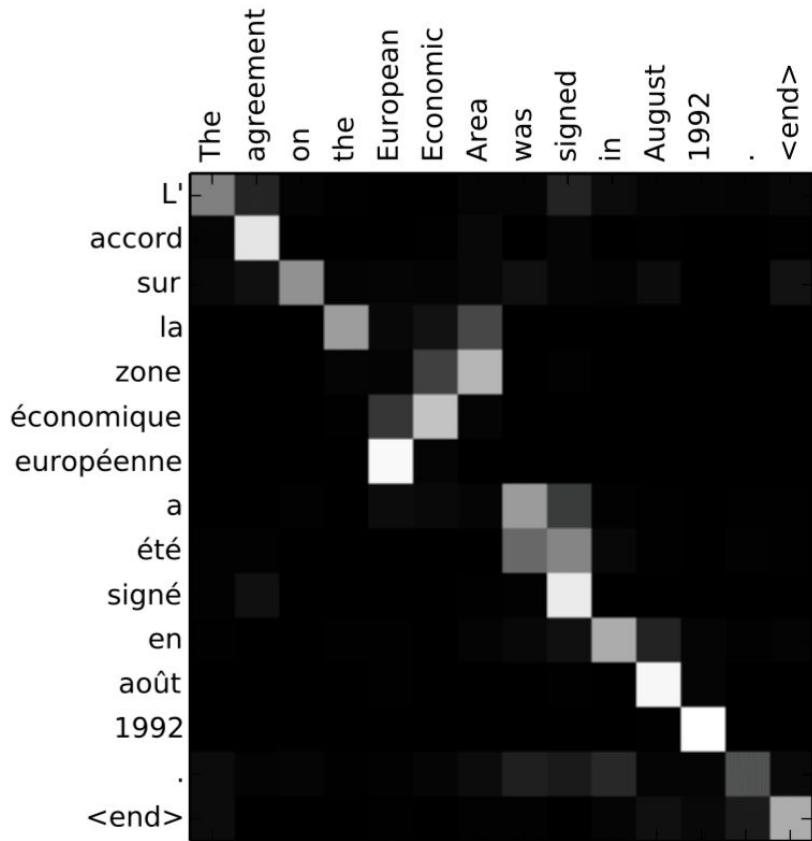
Bahdanau model



Luong model



Alignment



Attention is ALL you need

	Seq2seq without attention	Seq2seq with attention	Transformer
processing within encoder	RNN/CNN	RNN/CNN	attention
processing within decoder	RNN/CNN	RNN/CNN	attention
decoder-encoder interaction	static fixed- sized vector	attention	attention

Что хотим от трансформера?

Encoder

Who is doing:

- all source tokens

What they are doing:

- look at each other
 - update representations
- repeat
N times

Decoder

Who is doing:

- target token at the current step

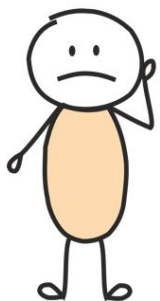
What they are doing:

- looks at previous target tokens
 - looks at source representations
 - update representation
- repeat
N times

Почему RNN будет работать хуже?

I arrived at the **bank** after crossing thestreet? ...river?

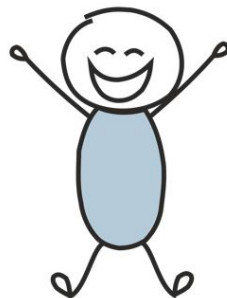
What does **bank** mean in this sentence?



I've no idea: let's wait
until I read the end

RNNs

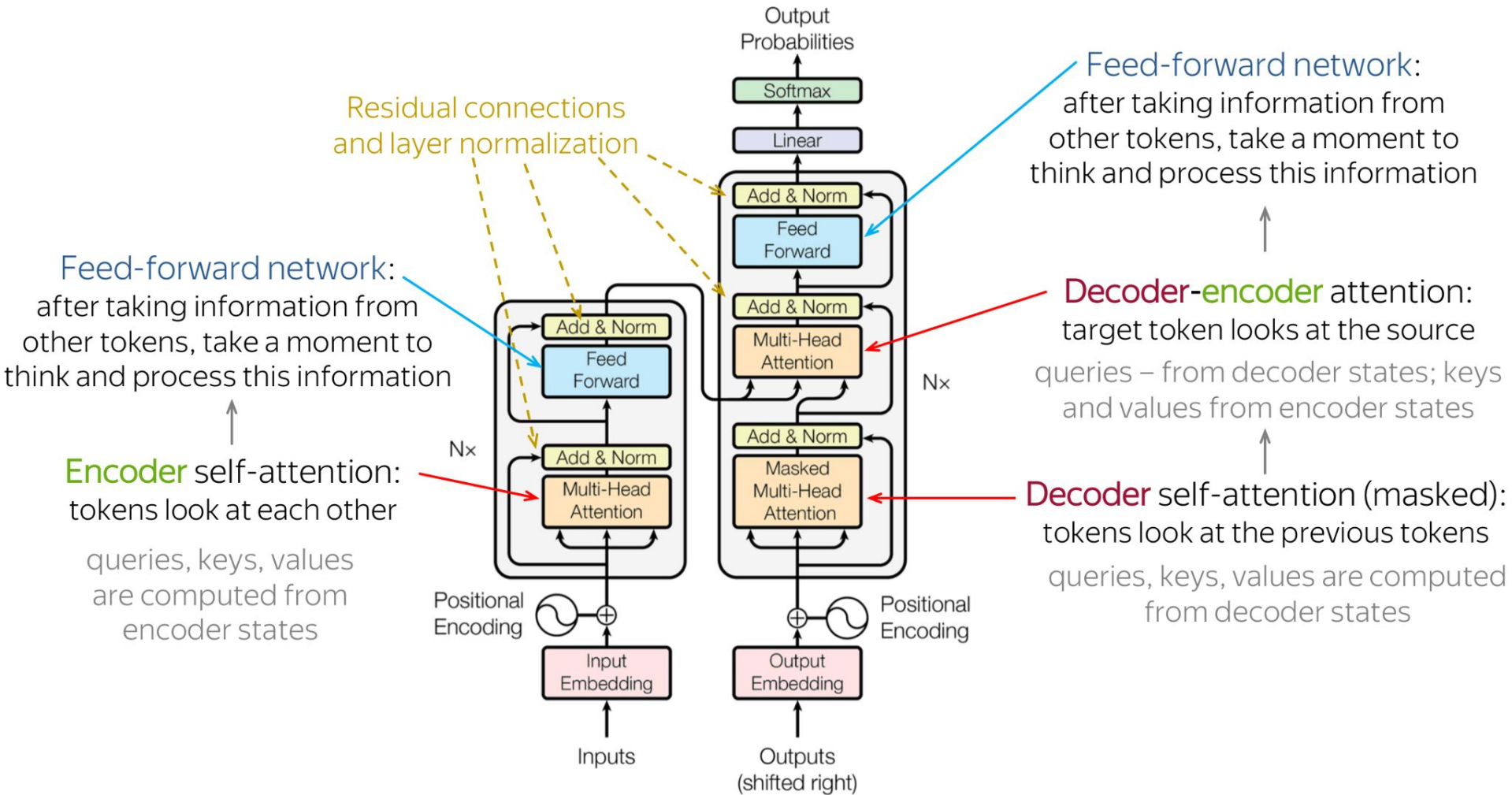
$O(N)$ steps to process a
sentence with length N



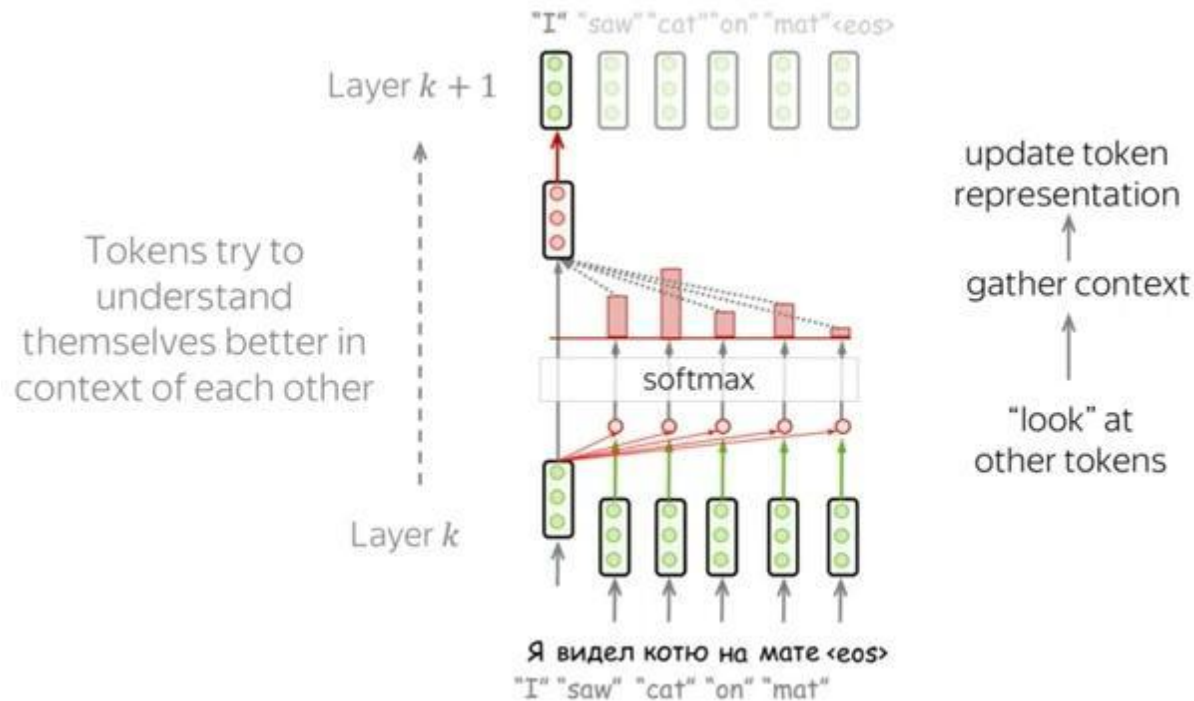
I don't need to wait - I
see all words at once!

Transformer

Constant number of steps



Self-attention



Self-attention

Each vector receives three representations (“roles”)

$$\begin{bmatrix} W_Q \end{bmatrix} \times \begin{bmatrix} \bullet \\ \bullet \\ \bullet \end{bmatrix} = \begin{bmatrix} \bullet \\ \bullet \end{bmatrix}$$

Query: vector **from** which the attention is looking

“Hey there, do you have this information?”

$$\begin{bmatrix} W_K \end{bmatrix} \times \begin{bmatrix} \bullet \\ \bullet \\ \bullet \end{bmatrix} = \begin{bmatrix} \bullet \\ \bullet \end{bmatrix}$$

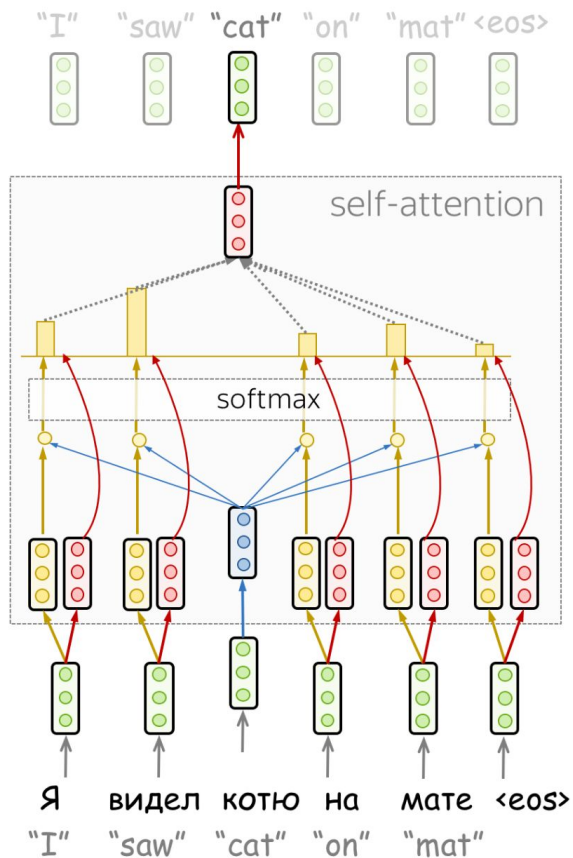
Key: vector **at** which the query looks to compute weights

“Hi, I have this information – give me a large weight!”

$$\begin{bmatrix} W_V \end{bmatrix} \times \begin{bmatrix} \bullet \\ \bullet \\ \bullet \end{bmatrix} = \begin{bmatrix} \bullet \\ \bullet \\ \bullet \end{bmatrix}$$

Value: their weighted sum is attention output

“Here’s the information I have!”

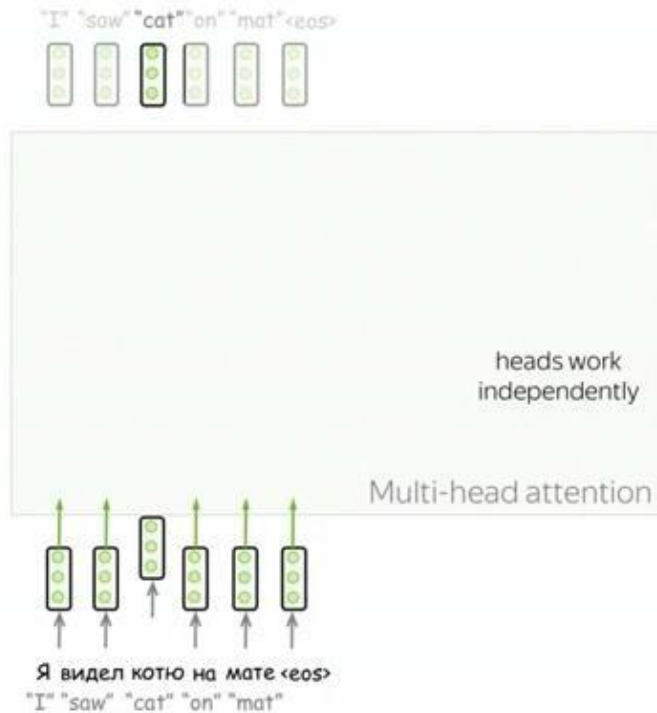


Self-attention

$$\textit{Attention}(\underset{\substack{\text{from}}}{q}, \underset{\substack{\text{to}}}{k}, v) = \overbrace{\textit{softmax}\left(\frac{qk^T}{\sqrt{d_k}}\right)}^{\text{Attention weights}} v$$

vector dimensionality of K, V

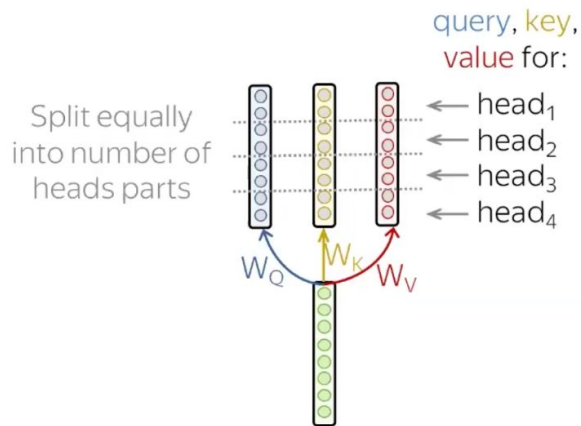
Multi-head attention



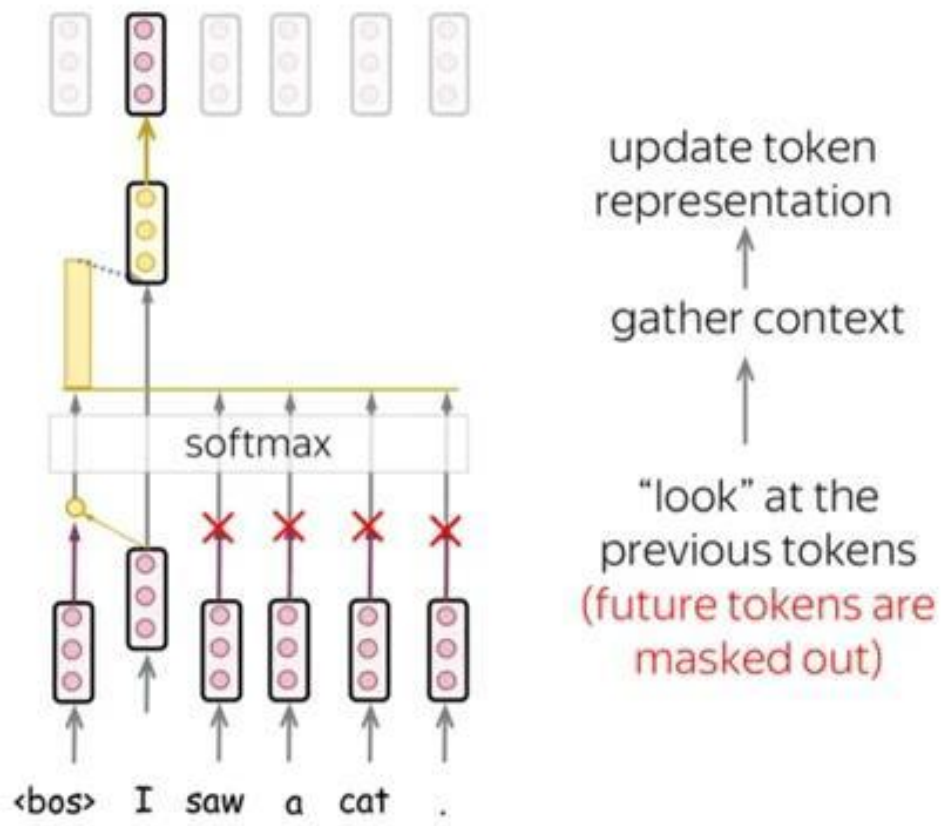
Multi-head attention

$$\text{MultiHead}(Q, K, V) = \text{Concat}(\text{head}_1, \dots, \text{head}_n)W_o,$$

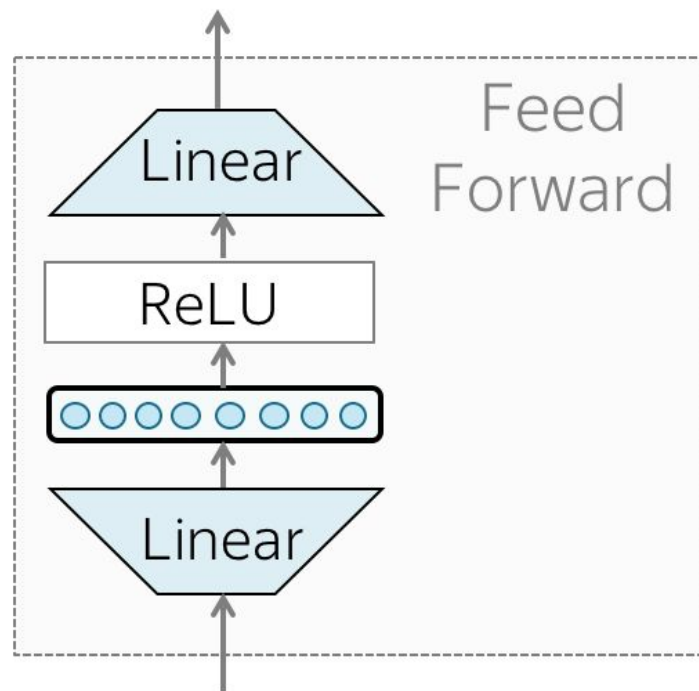
$$\text{head}_i = \text{Attention}(QW_Q^i, KW_K^i, VW_V^i)$$



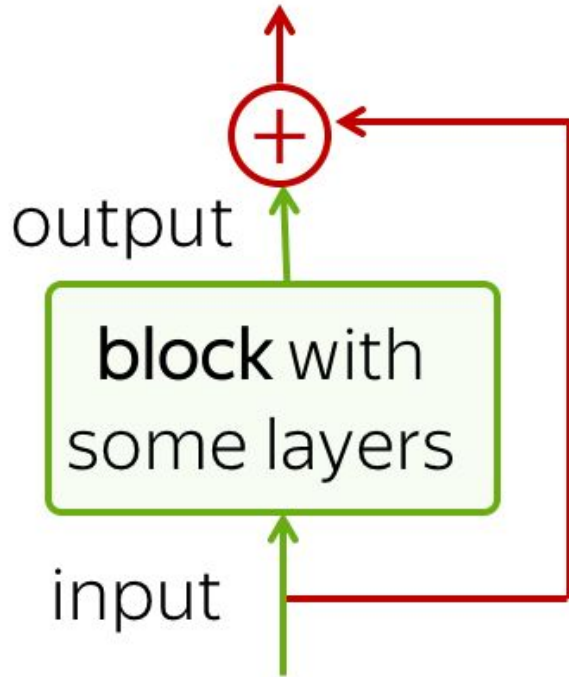
Masked attention



FFN

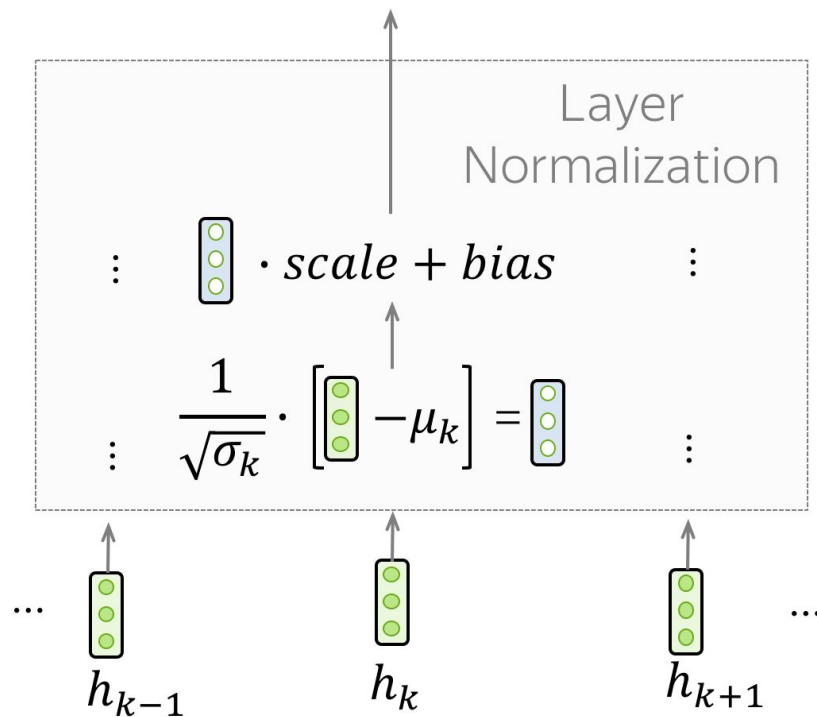


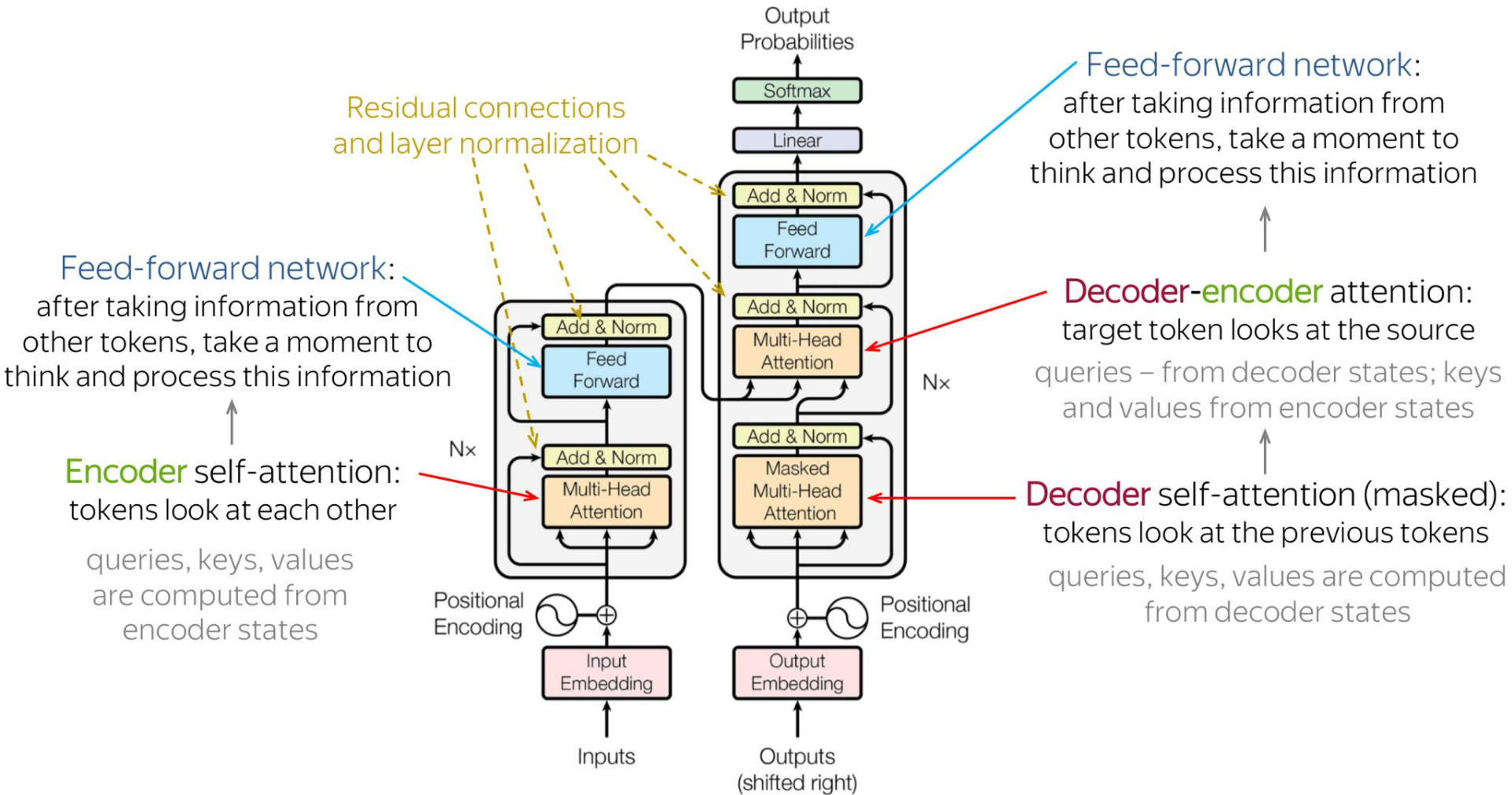
Residual connection



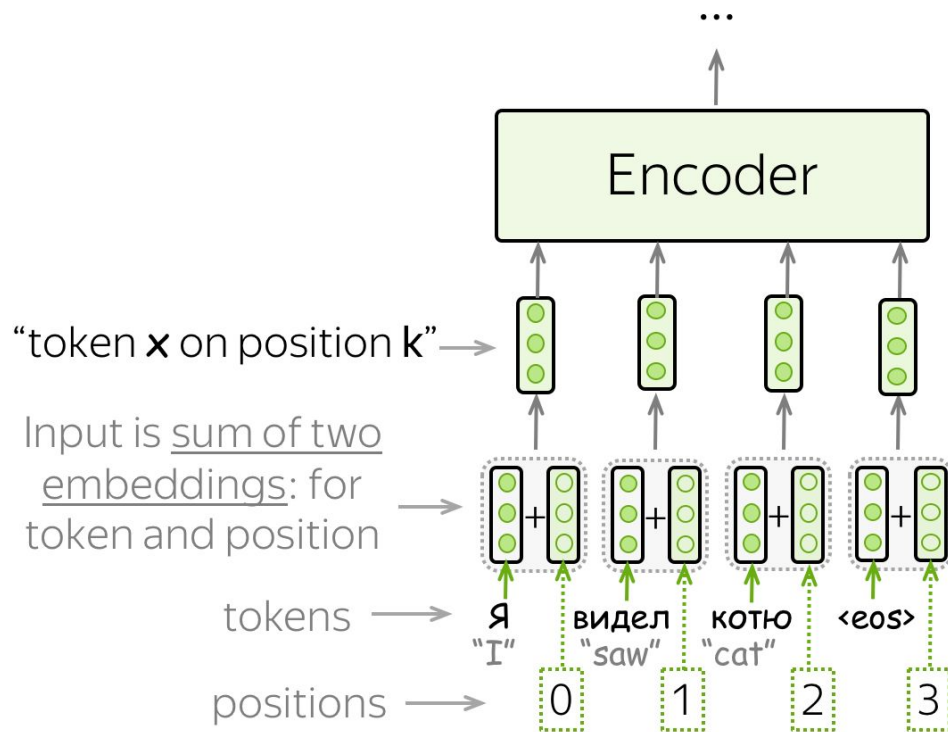
Residual connection:
add a block's input to
its output

Layer norm





Positional encoding



$$\text{PE}_{pos,2i} = \sin(pos/10000^{2i/d_{model}}),$$

$$\text{PE}_{pos,2i+1} = \cos(pos/10000^{2i/d_{model}})$$

BPE (byte-pair encoding)

Initial vocabulary:

characters



Split each word
into characters

Words in the data:

word	count
------	-------

c a t	4
-------	---

m a t	5
-------	---

m a t s	2
---------	---

m a t e	3
---------	---

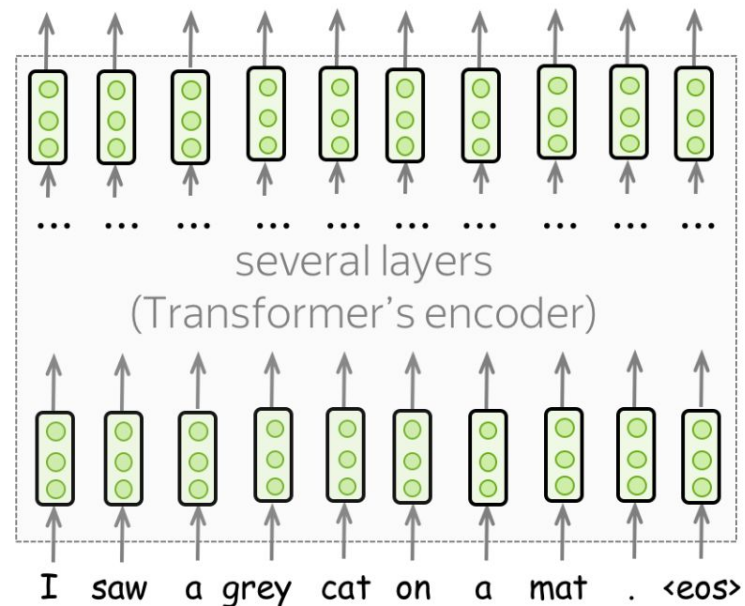
a t e	3
-------	---

e a t	2
-------	---

Current merge table:

(empty)

BERT Bidirectional Encoder Representations from Transformers



Model architecture:

- Transformer's encoder

What is special about it:

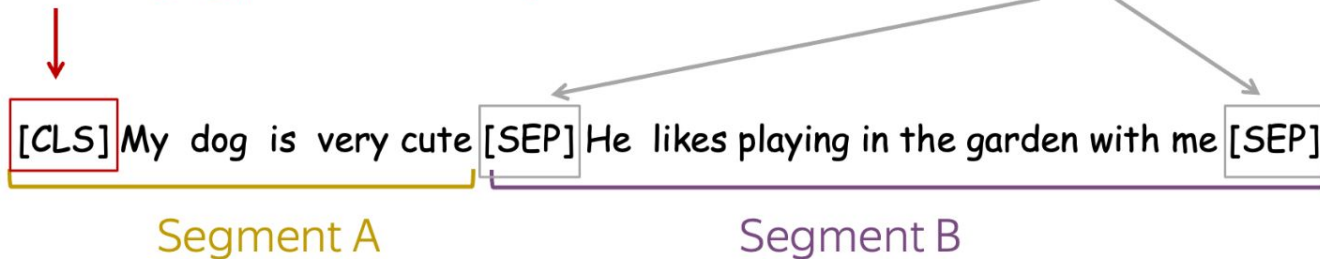
- Training objectives
 - MLM: Masked language modeling
 - NSP: Next sentence prediction
- The way it is used
 - No task-specific models

Input

[CLS]: Special token

- Training time: predict if sentences are consecutive or not (Next Sentence Prediction /NSP objective)
- Test time: downstream tasks (e.g., classification)

[SEP]: Special token-separator



Training on pairs of sentences: either consecutive or random (50%/50%)

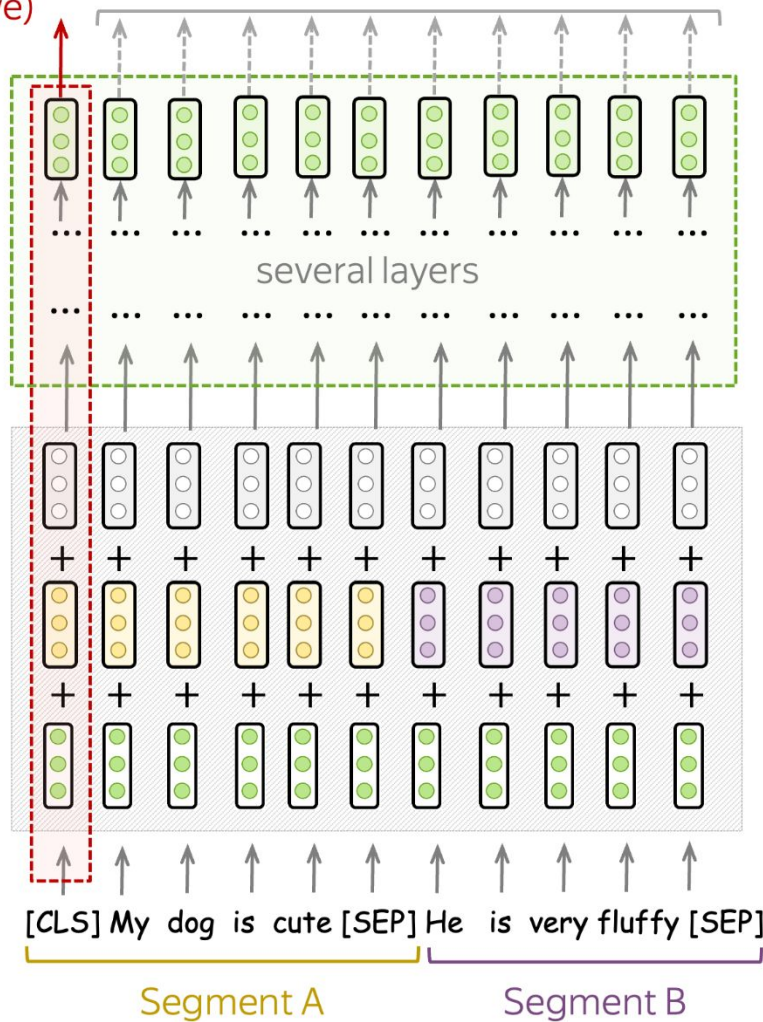
Training time: predict if sentences
are consecutive (NSP objective)
Test time: classification

Training time: MLM objective

Model
(Transformer
encoder)

Input

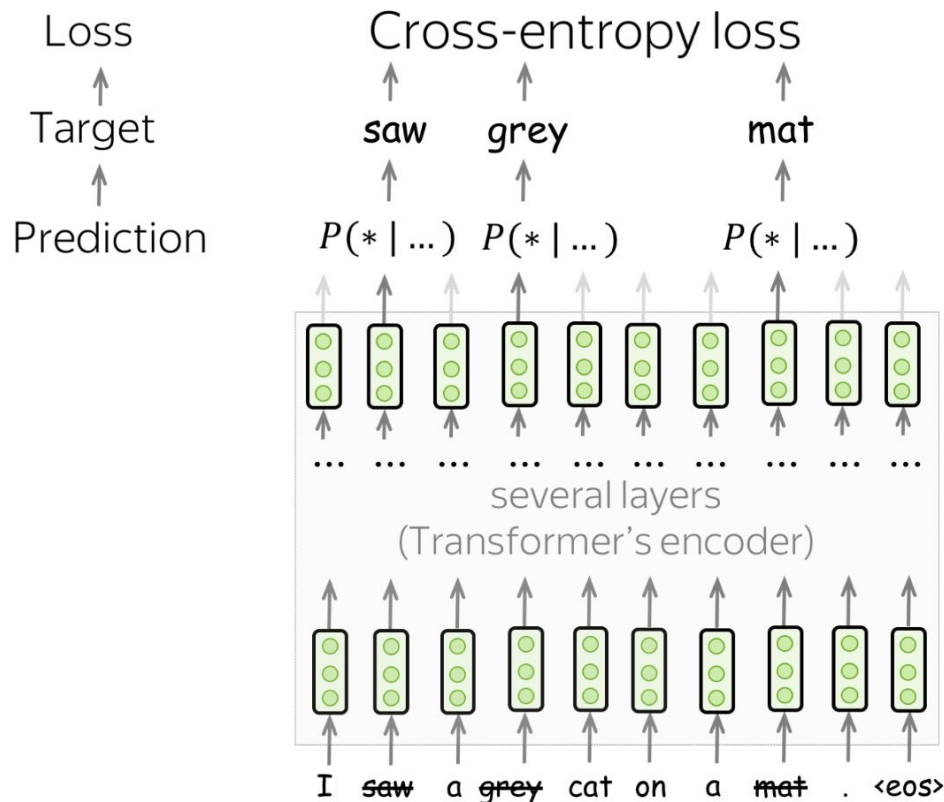
Training on pairs of
sentences: either
consecutive or
random (50%/50%)



positions
0 1 2 3 4 ...

segments
A A A A A A B B B B B

tokens
[CLS] My dog is ...



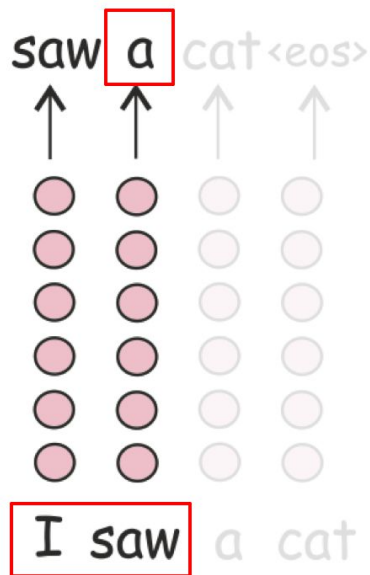
At each training step:

- pick randomly 15% of tokens
- replace each of the chosen tokens with something
- predict original chosen tokens

- **[MASK]**, with $p = 80\%$
- Random token, with $p = 10\%$
- Original token, with $p = 10\%$

Language Modeling

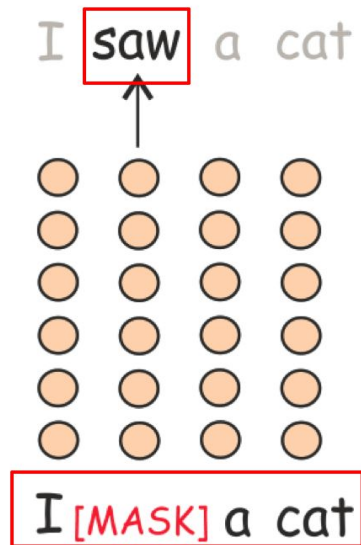
- Target: next token
- Prediction: $P(* | \mathbf{I} \text{ saw})$



left-to-right, does
not see future

Masked Language Modeling

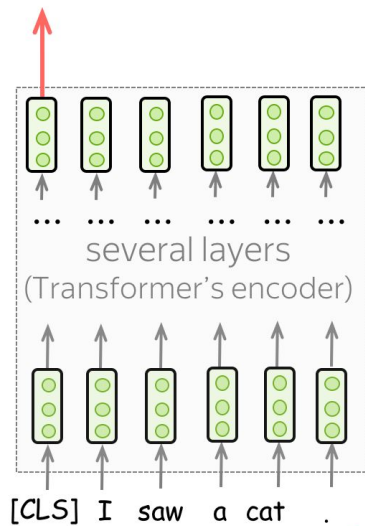
- Target: current token (the true one)
- Prediction: $P(* | \mathbf{I} [\text{MASK}] \text{ a cat})$



sees the whole text, but
something is corrupted

Sequence classification

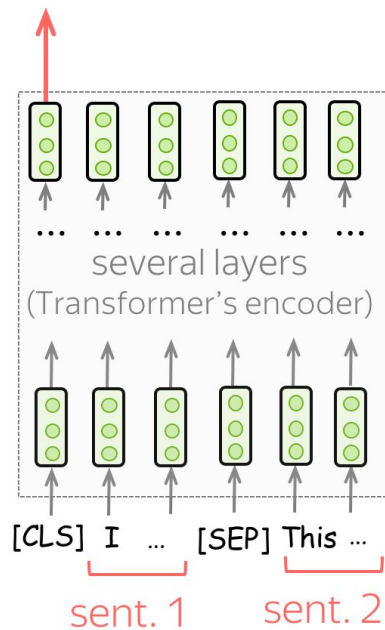
class label



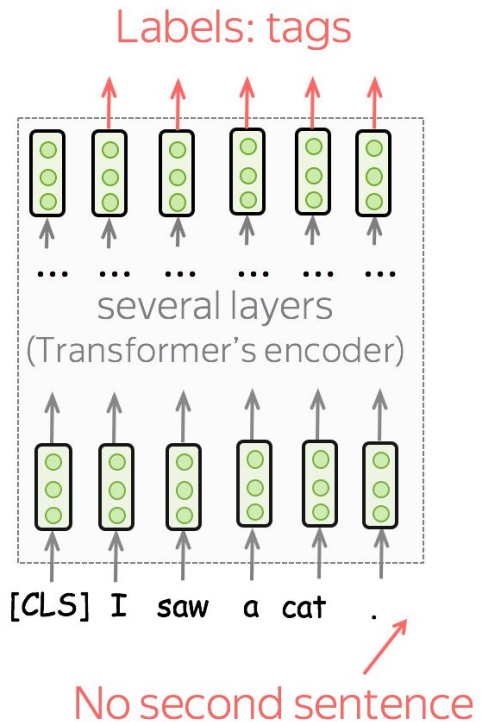
No second sentence!

Relevance (e.g.)

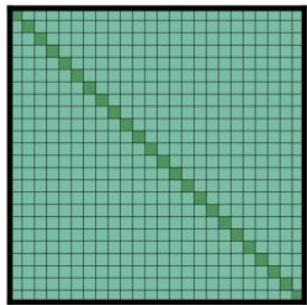
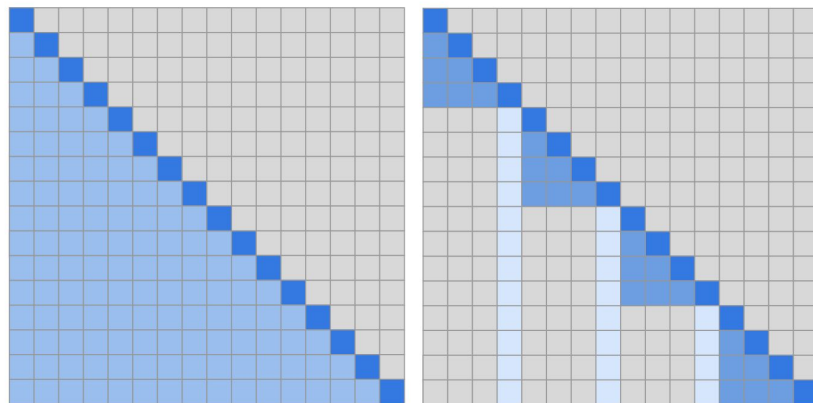
class label



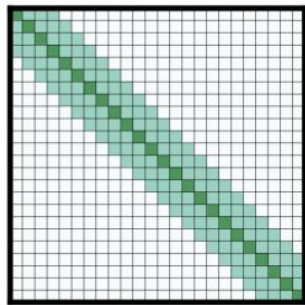
NER (named entity recognition)



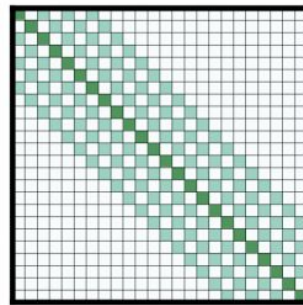
Разные маски attention



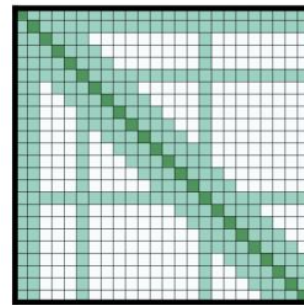
(a) Full n^2 attention



(b) Sliding window attention

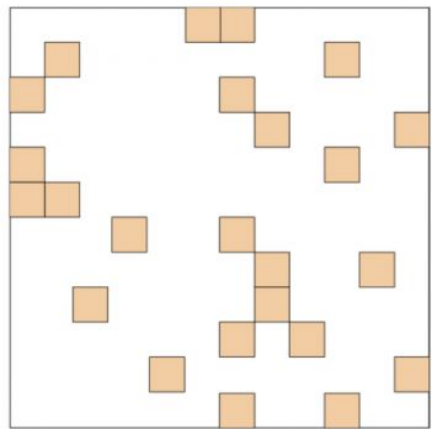


(c) Dilated sliding window

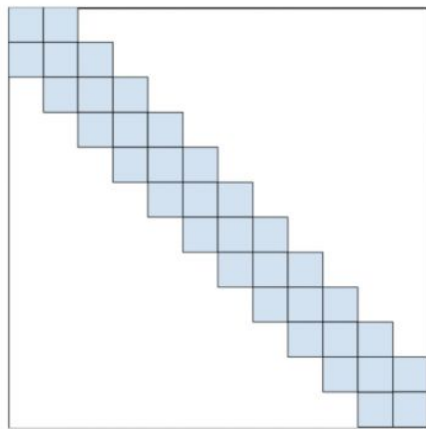


(d) Global+sliding window

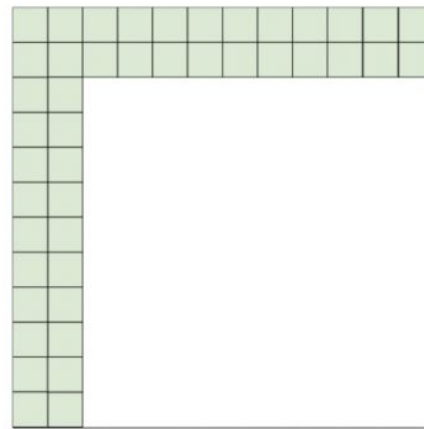
Разные маски attention



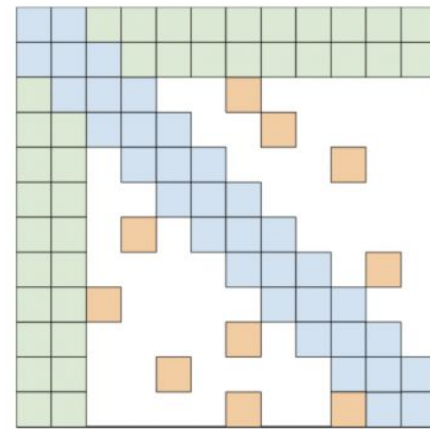
(a) Random attention



(b) Window attention



(c) Global Attention



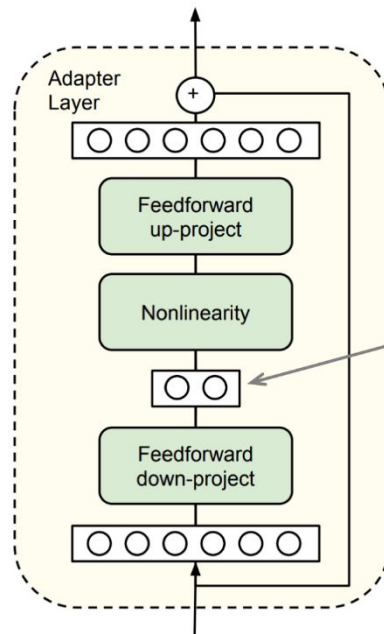
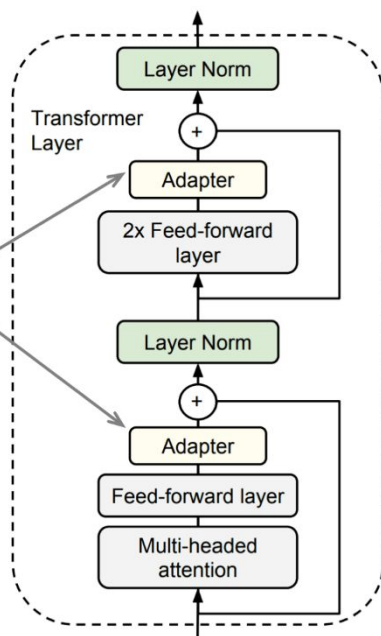
(d) BIGBIRD

Модификации подсчёта внимания

$$\text{softmax}(Q_{n \times d} K_{n \times d}^T) V_{n \times d} \approx Q'_{n \times r} (K'^T_{n \times r} V_{n \times d})$$

Адаптеры

Only these are trained,
everything else is fixed and
is the same for all tasks



Small hidden size, i.e.
an adaptor has only a
few parameters
(which is good!)

LoRA (Low Rank Adaptation)

